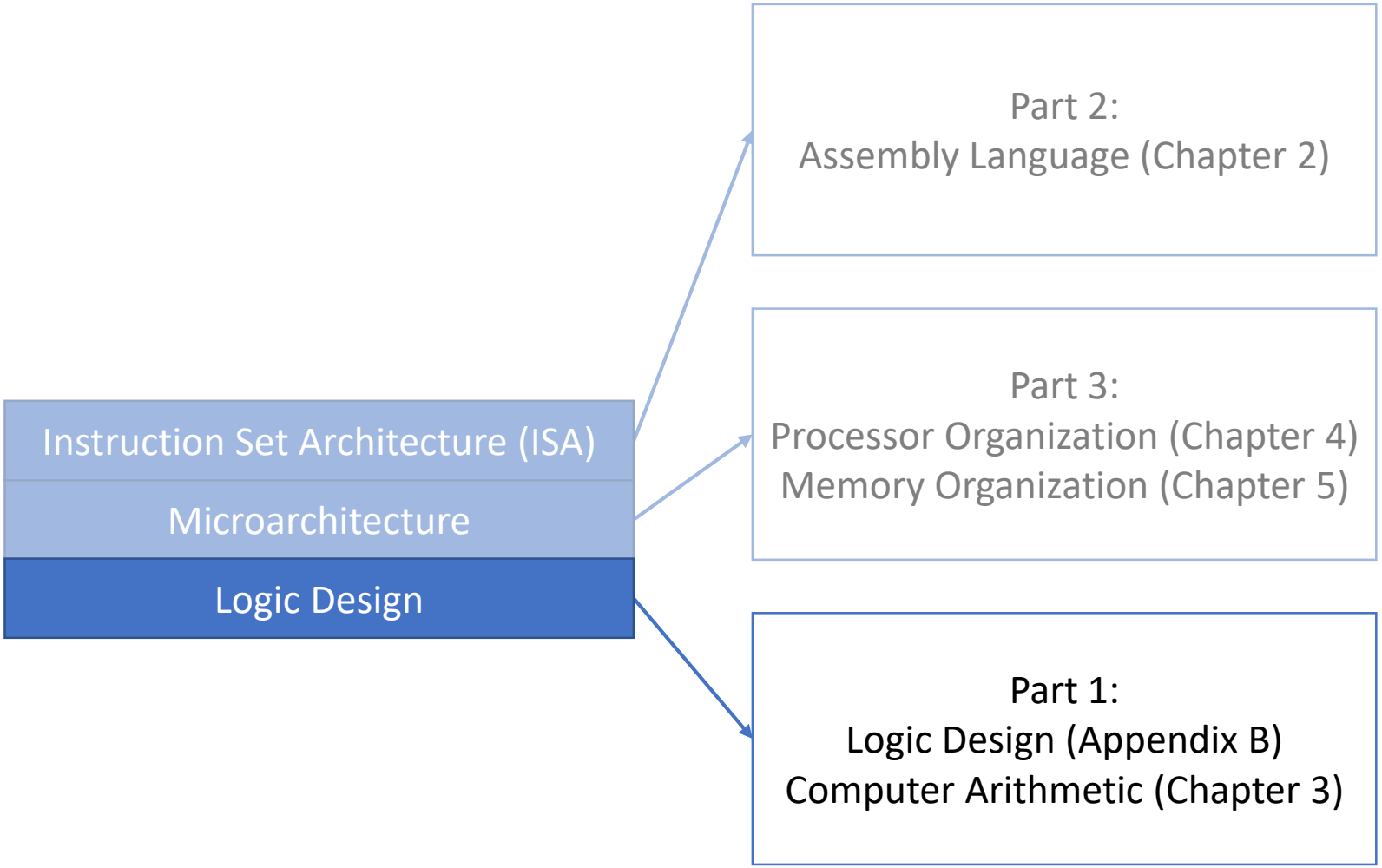
Lecture 12: Multiplication Hardware

CMPS 221 – Computer Organization and Design

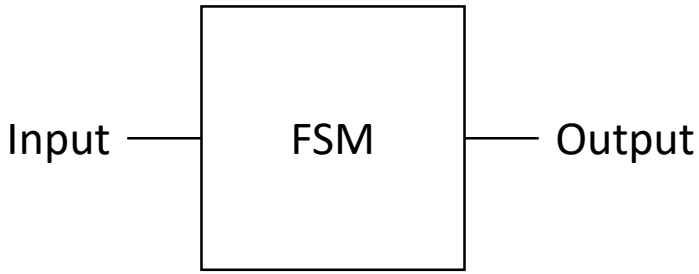
Slides by Mahmoud Bdeir and Izzat El Hajj

Last Time: Finite State Machines



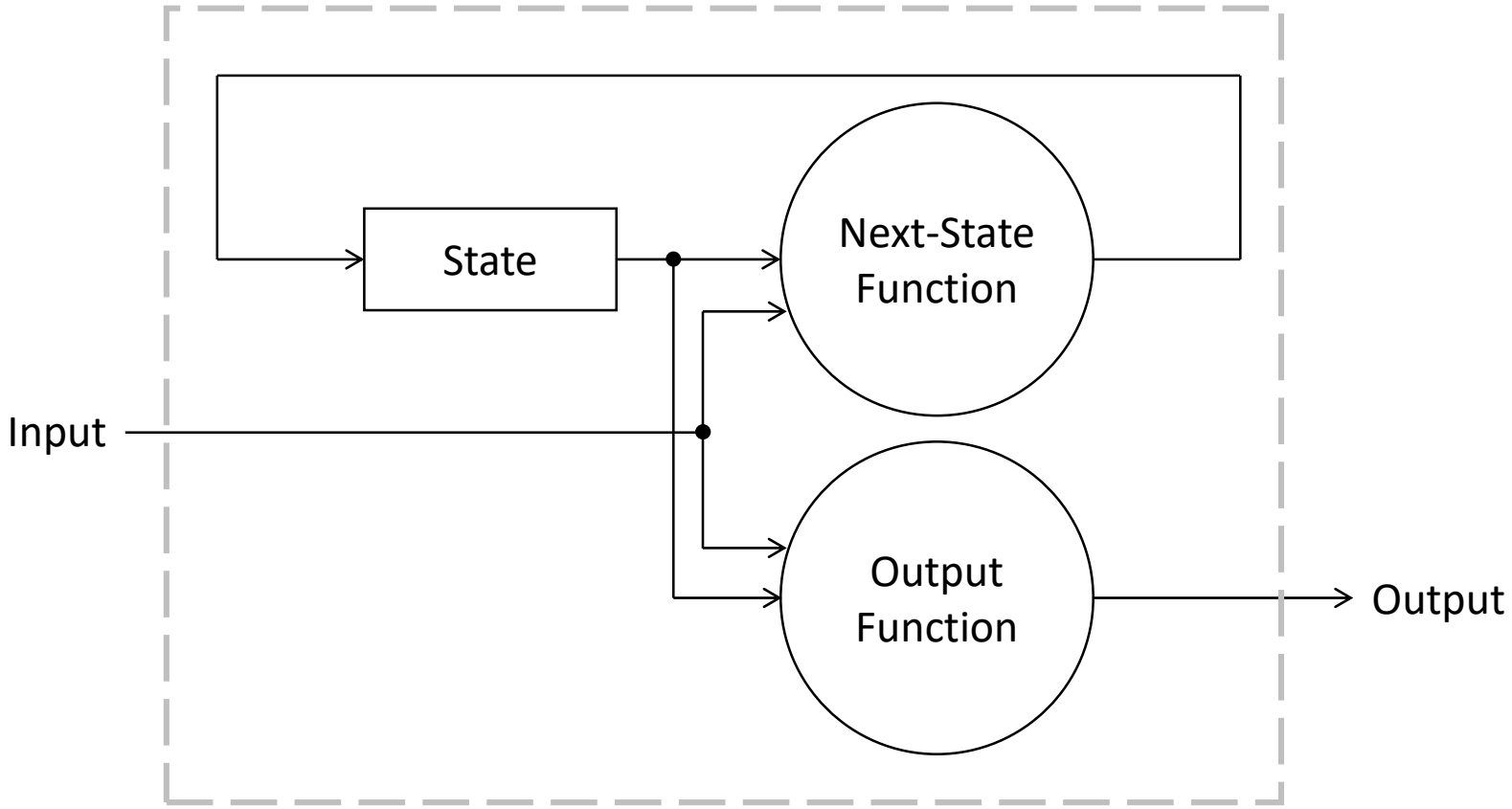
Finite State Machine

- A **finite state machine** (FSM) is a logic block consisting of inputs, outputs, and internal state
  - It includes an **output function** that computes the output from the inputs and current state
  - It includes a **next-state function** that computes a next state from the inputs and current state

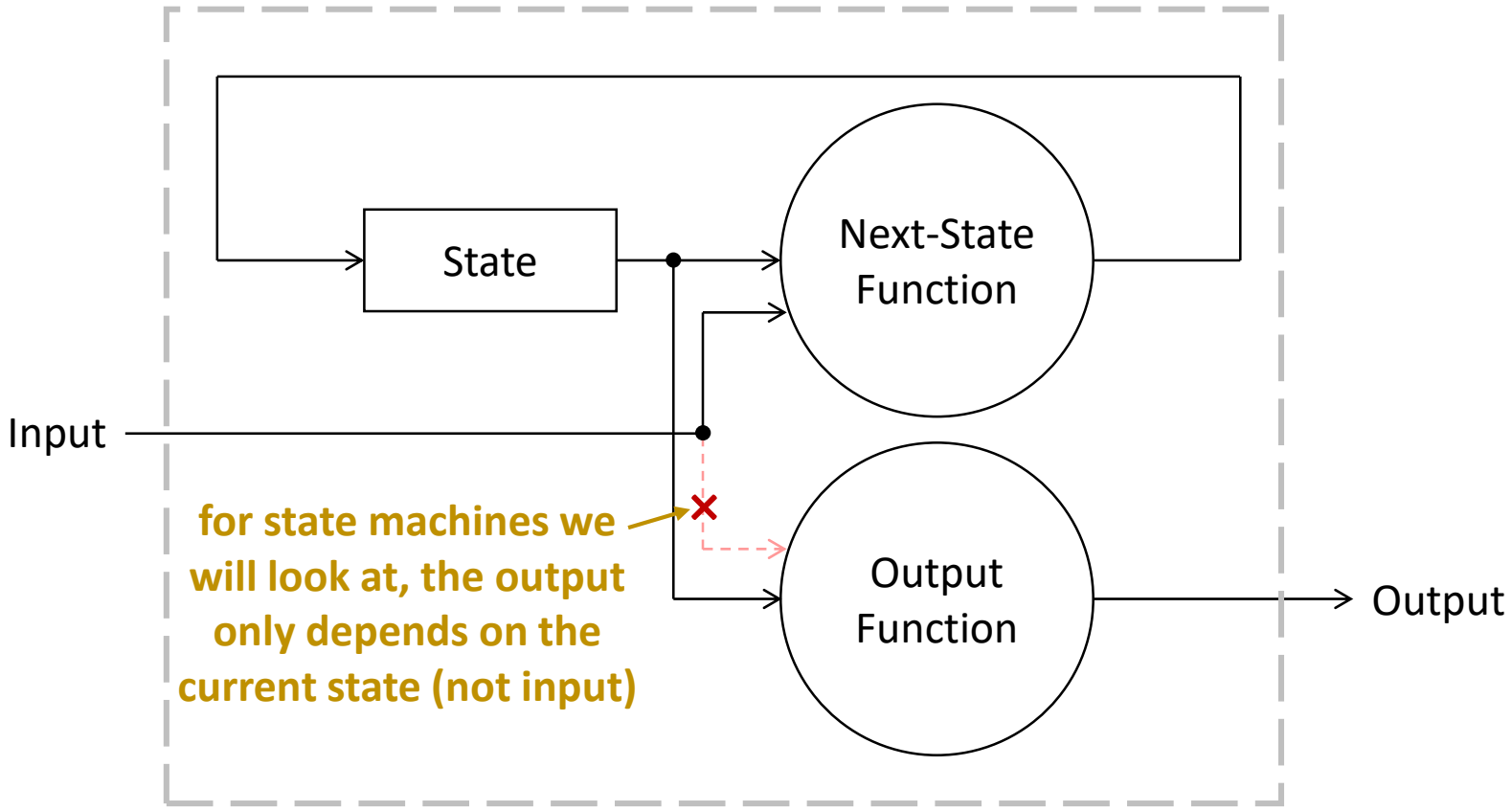


  - FSMs are used in computer architecture for building **controllers** which orchestrate a CPU's work, telling each of the other units what to do

Finite State Machine

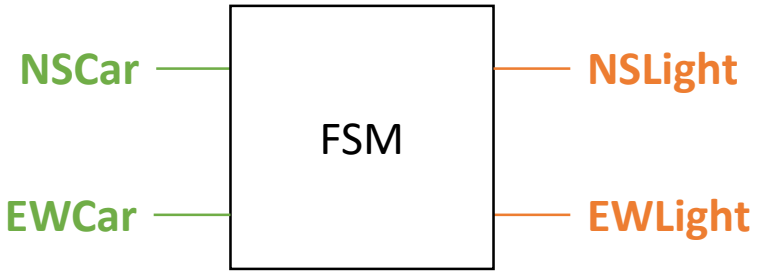


Finite State Machine



FSM Example

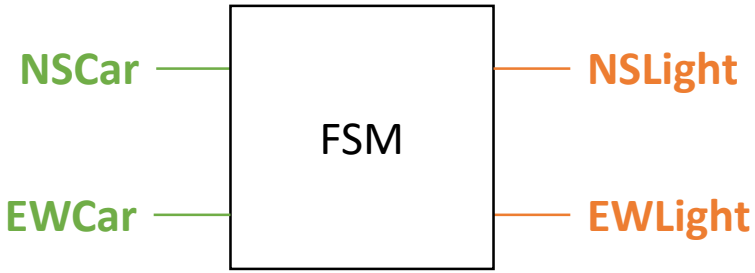
- Example: a finite state machine that controls a traffic signal at an intersection



  - Inputs:
    - **NSCar**: there is a car on the north-south road
    - **EWCar**: there is a car on the east-west road
  - Outputs:
    - **NSLight**: turn the north-south light green
    - **EWLight**: turn the east-west light green

FSM Example

- Example: a finite state machine that controls a traffic signal at an intersection



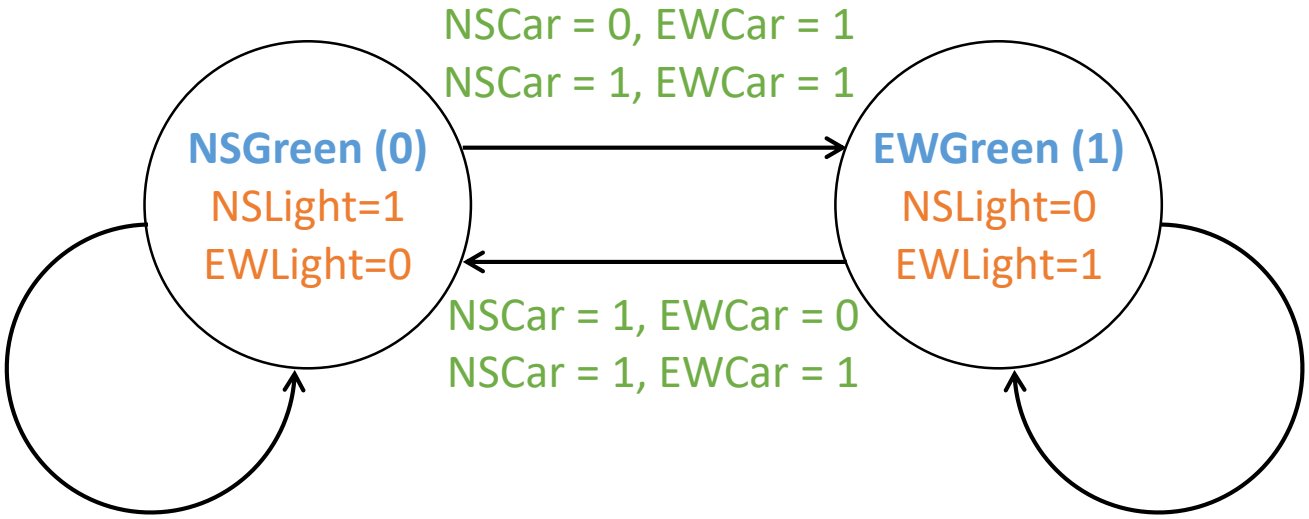
  - Rules:
    - Rule #1: If a car is present on one road but not the other, turn green on the road where the car is
    - Rule #2: If a car is not present on either roads, stay green on the road that is currently green
    - Rule #3: If a car is present on both roads, turn green on the road that is not currently green

State Diagram

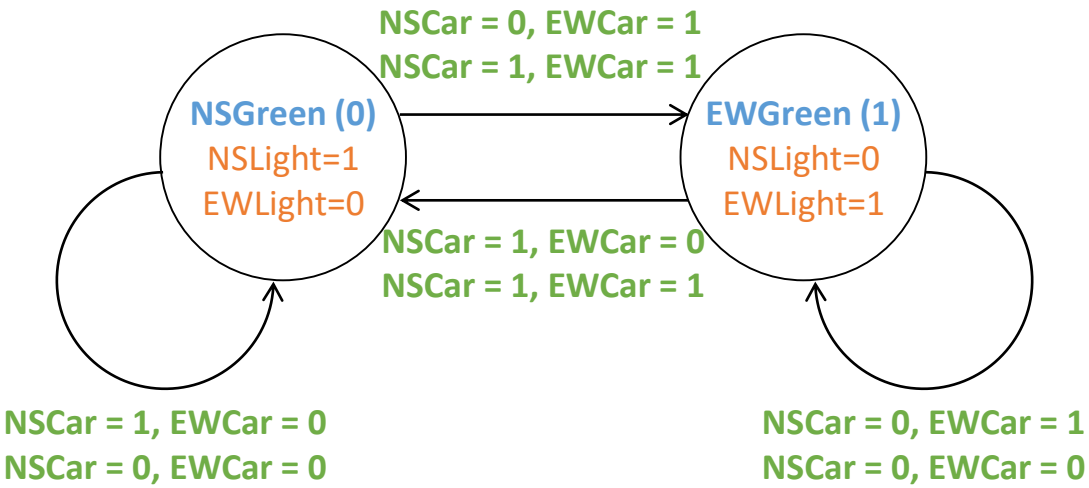
Inputs:
NSCar
EWCar

Outputs:
NSLight
EWLight

States:
NSGreen
EQGreen



Truth Table and Logic Equations



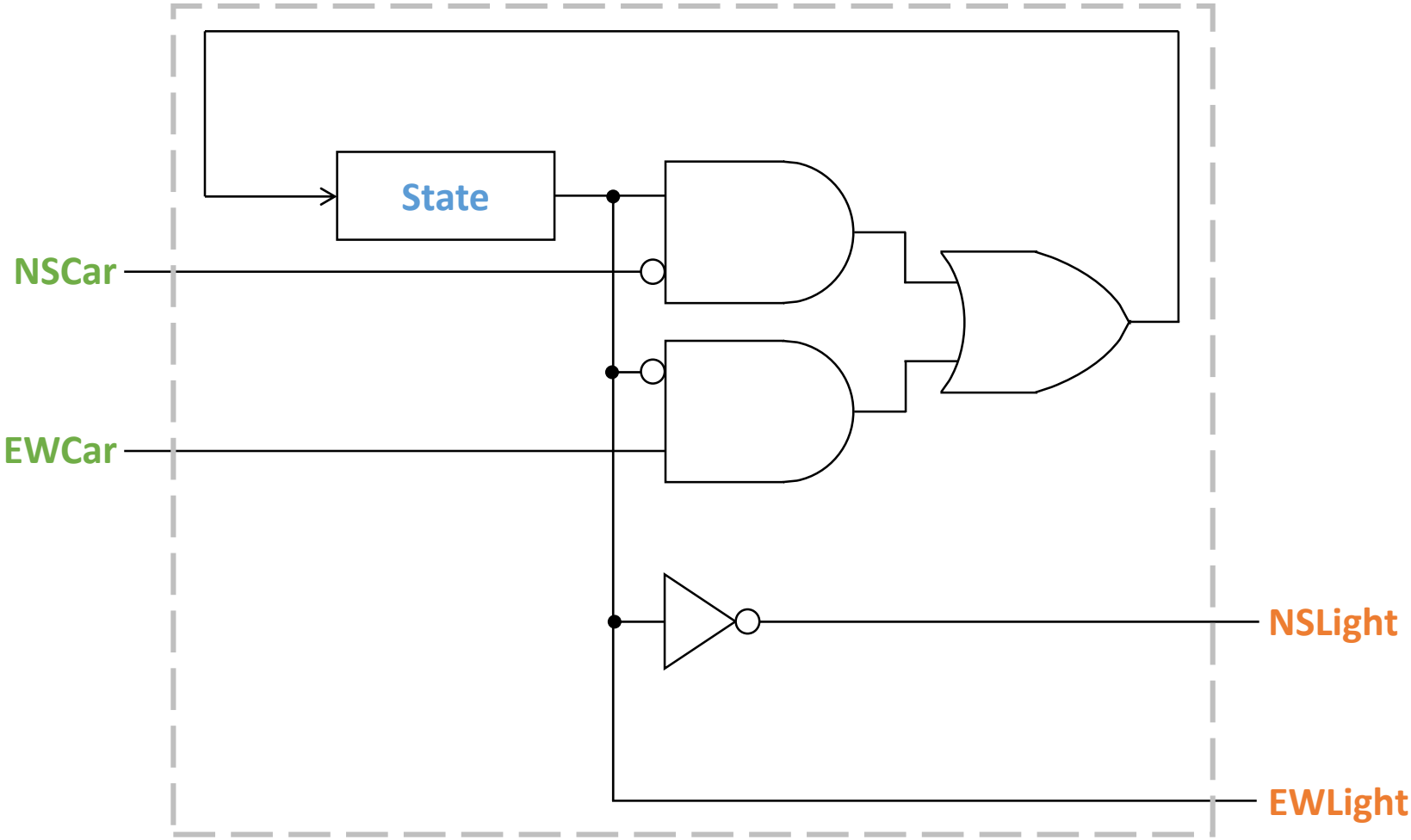
$State_{current}$	Outputs	
	NSLight	EWLight
0	1	0
1	0	1

$State_{current}$	Inputs		$State_{next}$
	NSCar	EWCar	
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Output Function: $NSLight = \overline{State_{current}}$ $\quad EWLight = State_{current}$

Next-State Function: $State_{next} = State_{current} \cdot \overline{NSCar} + \overline{State_{current}} \cdot EWCar$

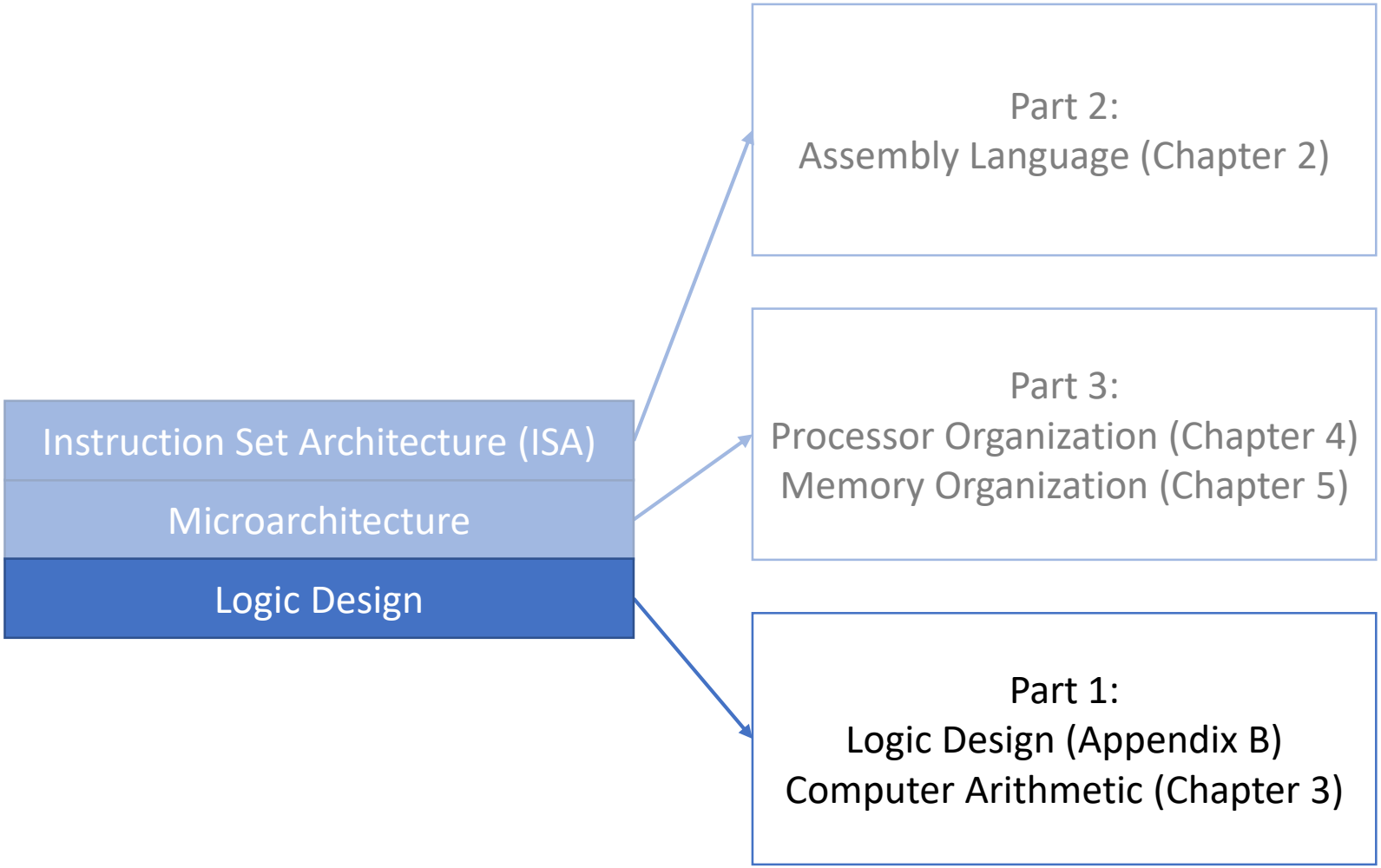
Logic Diagram



Output Function: $NSLight = \overline{State_{current}}$ $\quad EWLight = State_{current}$

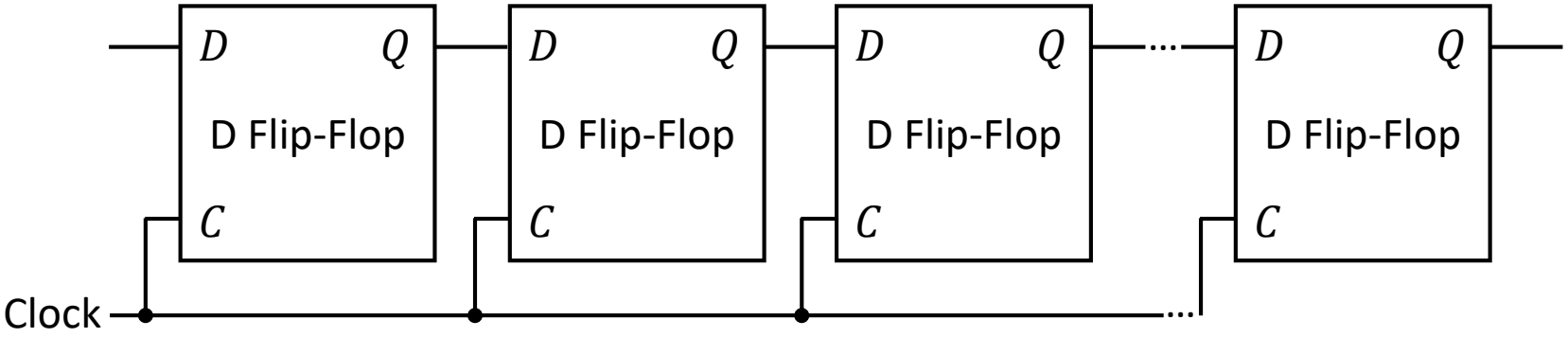
Next-State Function: $State_{next} = State_{current} \cdot \overline{NSCar} + \overline{State_{current}} \cdot EWCar$

Today: Multiplication Hardware



Shift Registers

- A **shift register** is an array of cascaded flip-flops sharing the same clock, where the output of one flip-flop is the input to the next flip-flop



- The resulting behavior is that the value stored in the register will shift by one position every cycle
  - Useful for implementing multiplication and division (and many other things)

Recall: Multiplication

```
Multiplicand:      1101
Multiplier:    ×   1011
               --------
                   1101
                  1101
                 0000
                1101
               --------
Product:       10001111
```

How to implement multiplication in hardware?

Implementing Multiplication

```
Multiplicand:      1101
Multiplier:    ×   1011
               --------
                   1101
                  1101
                 0000
                1101
               --------
Product:       10001111
```

product of two 4-bit operands
can have up to 8 bits
(use an 8-bit register for product)

Implementing Multiplication

```
Multiplicand:      1101
Multiplier:    ×   1011
               --------
               00001101
               00011010
               00000000
               01101000
               --------
Product:       10001111
```

multiplication of two 4-bit numbers can be done as an addition of four 8-bit numbers
(use an 8-bit adder)

Implementing Multiplication

Multiplicand: 1101
Multiplier: × 1011

```
--------
0000 1101      00000000
0001 1010
0000 0000
0110 1000
```

Product:

Implementing Multiplication

Multiplicand: 1101

Multiplier: × 1011

```
--------
0000 1101  -->[+]<--  00000000
000 1101 0            00001101
0000 0000
01101000
```

Product:

addition of four 8-bit numbers can be done as a sequence four additions of two 8-bit numbers
(use an 8-bit adder with 2 inputs 4 times)

Implementing Multiplication

Multiplicand: 1101

Multiplier: × 1011

```
--------
00001101 -> (+) <- 00000000
000110100 -> (+) <- 00001101
00000000         00100111
01101000
```

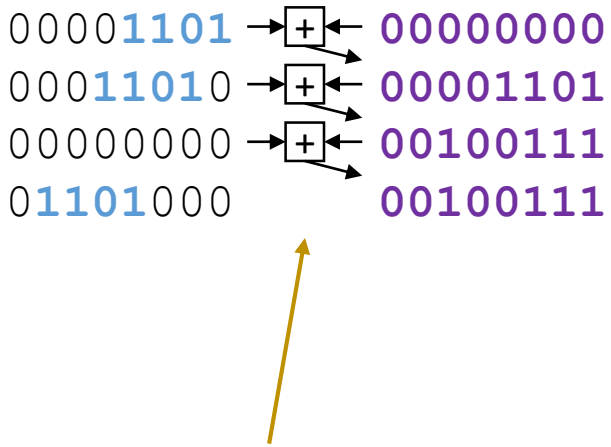
Product:

addition of four 8-bit numbers can be done as a sequence four additions of two 8-bit numbers
(use an 8-bit adder with 2 inputs 4 times)

Implementing Multiplication

Multiplicand: 1101

Multiplier: × 1011



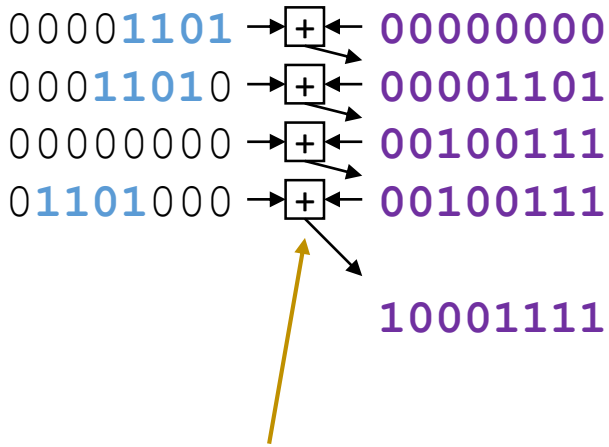
Product:

addition of four 8-bit numbers can be done as a sequence four additions of two 8-bit numbers
(use an 8-bit adder with 2 inputs 4 times)

Implementing Multiplication

Multiplicand: 1101

Multiplier: × 1011



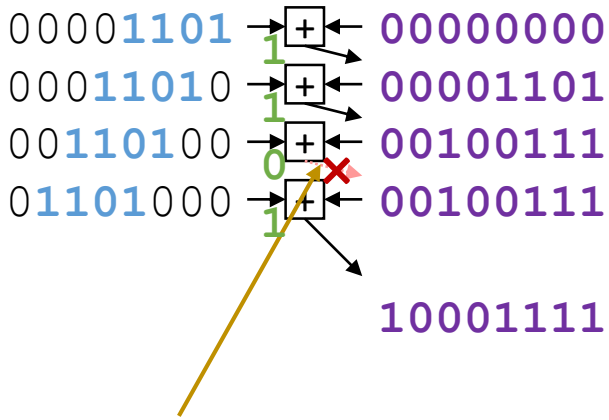
Product: 10001111

addition of four 8-bit numbers can be done as a sequence four additions of two 8-bit numbers
(use an 8-bit adder with 2 inputs 4 times)

Implementing Multiplication

Multiplicand: 1101

Multiplier: × 1011

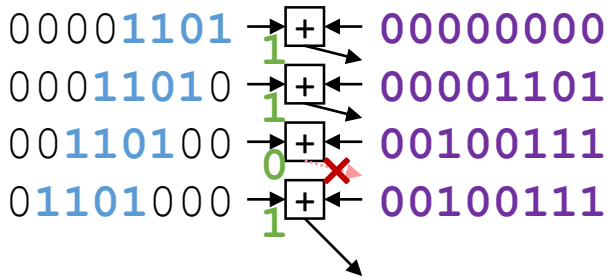


Product: 10001111

rather than multiply multiplicand with 0 and add, simply use 0 to disable updating the product with the sum

Implementing Multiplication

Multiplicand: 1101
Multiplier: × 1011



Product: 10001111

Overall approach:

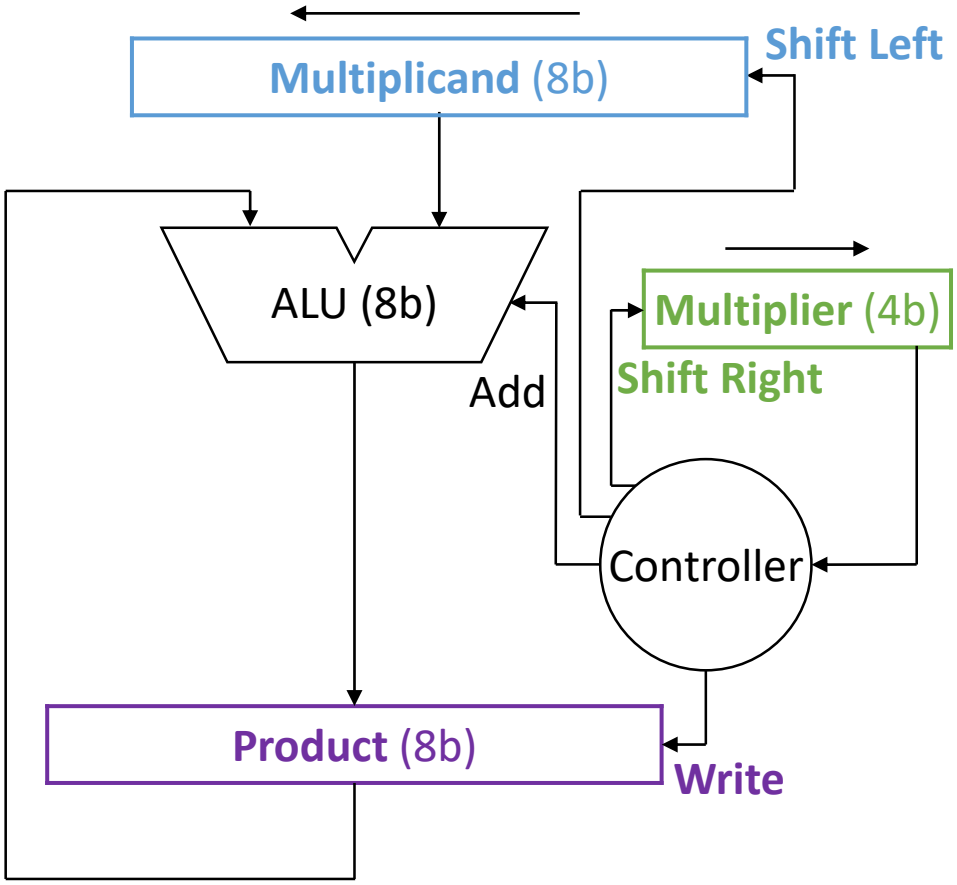
- Shift **multiplicand** left on every cycle and add it to **product**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

Implementing Multiplication

Multiplicand: 1101
Multiplier: × 1011

```
--------
00001101 + 00000000   (1)
000110100 + 00001101  (1)
001101000 + 00100111  (0)
01101000 + 00100111   (1)
```

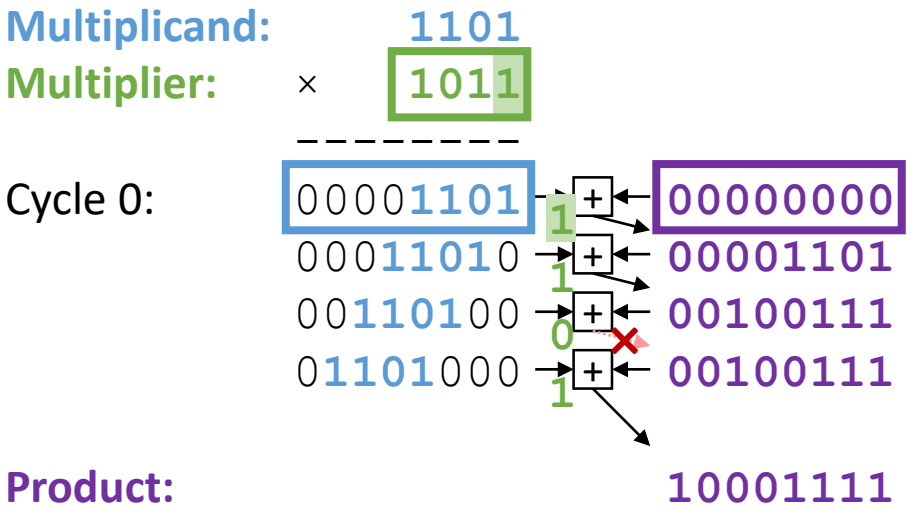
Product: 10001111

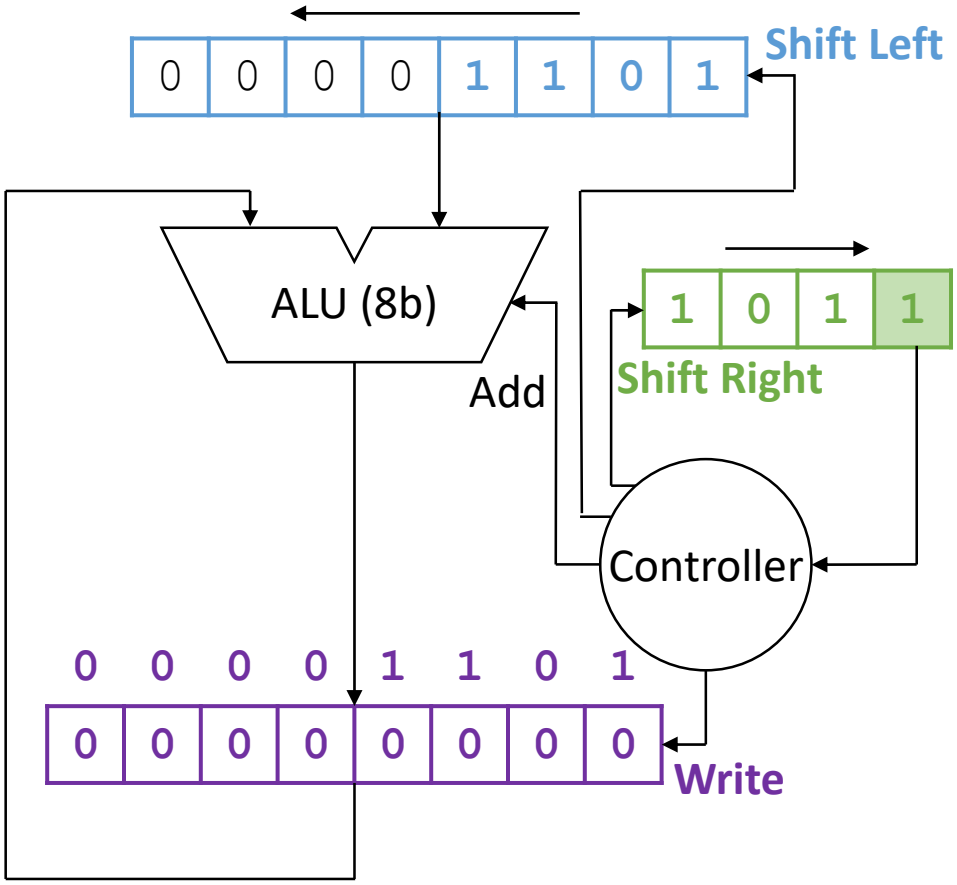


Overall approach:

- Shift **multiplicand** left on every cycle and add it to **product**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

Implementing Multiplication

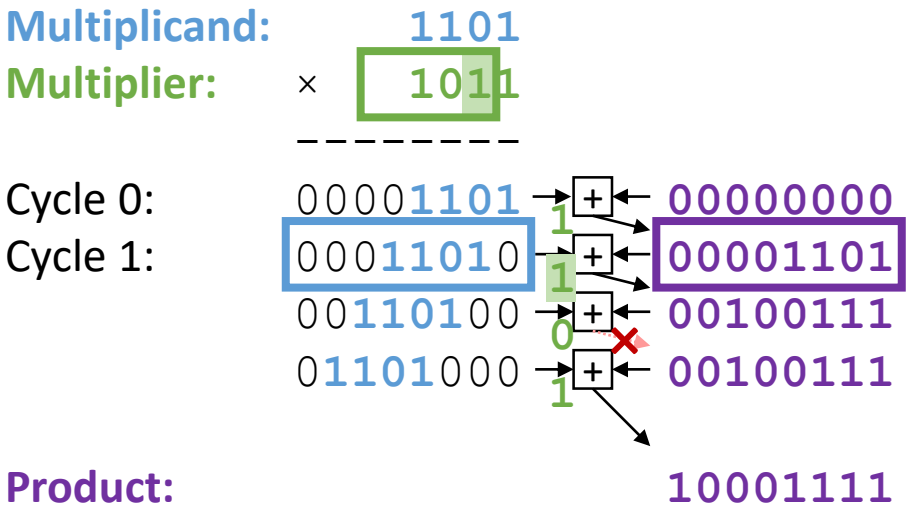


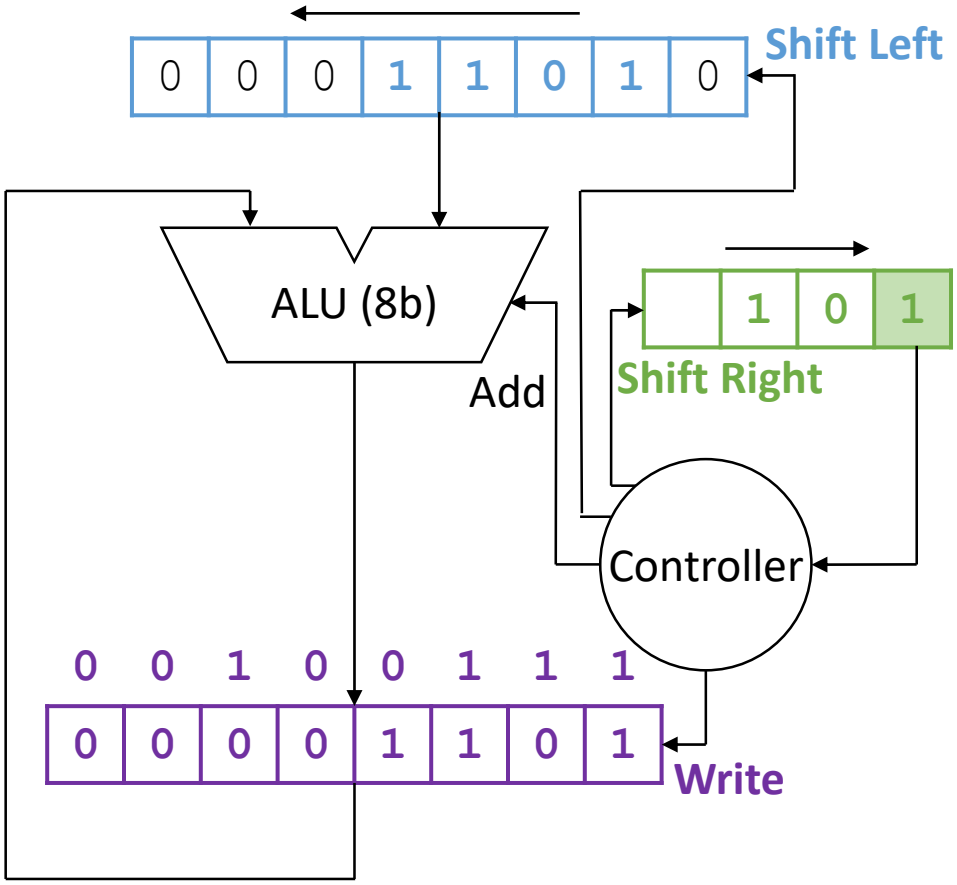


Overall approach:

- Shift **multiplicand** left on every cycle and add it to **product**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

Implementing Multiplication

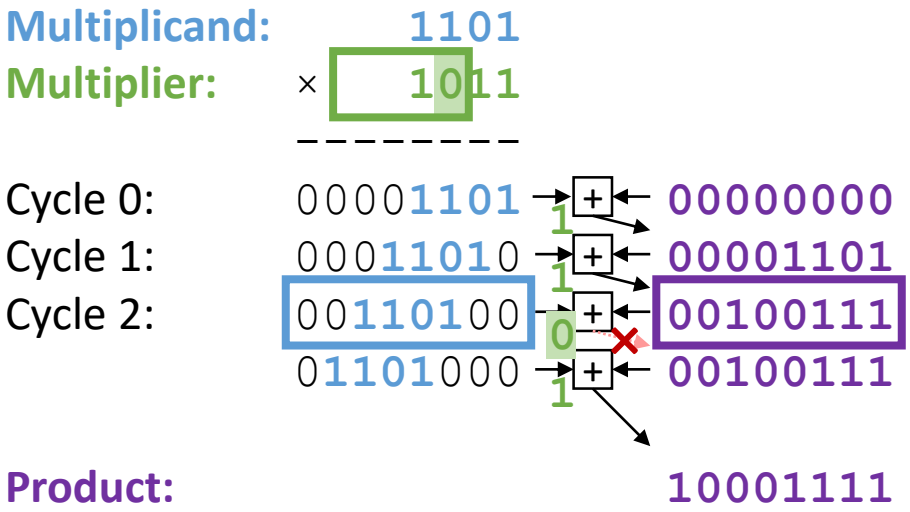


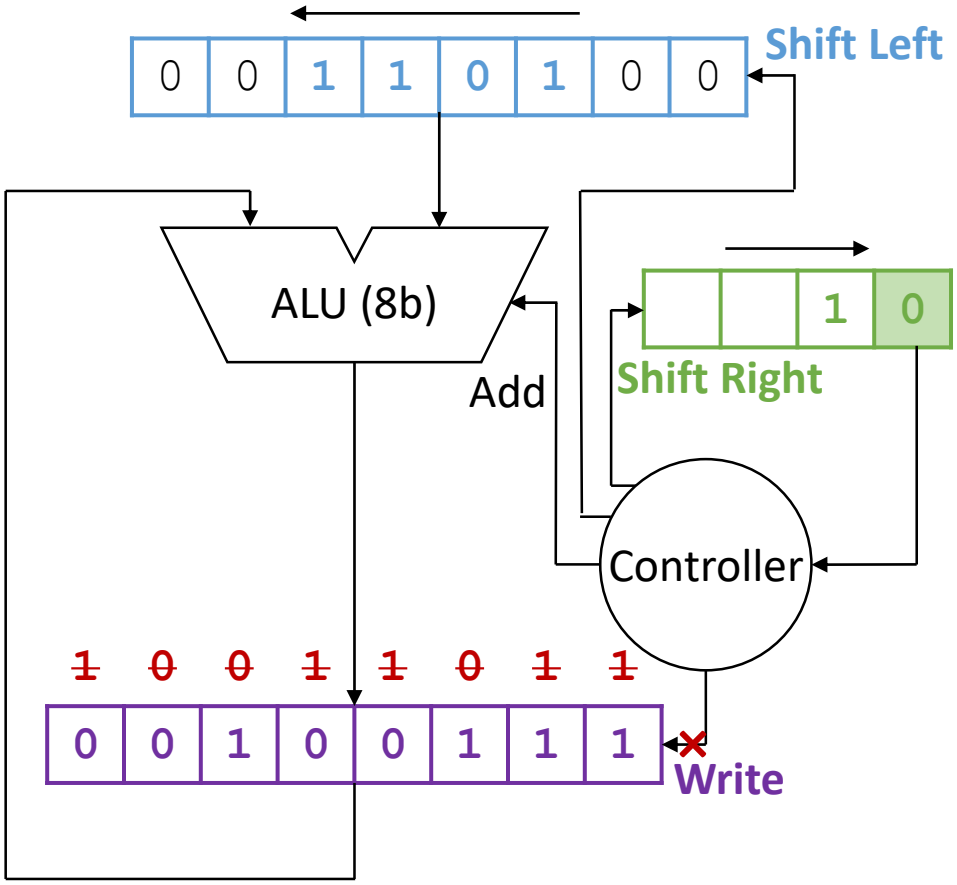


Overall approach:

- Shift **multiplicand** left on every cycle and add it to **product**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

Implementing Multiplication

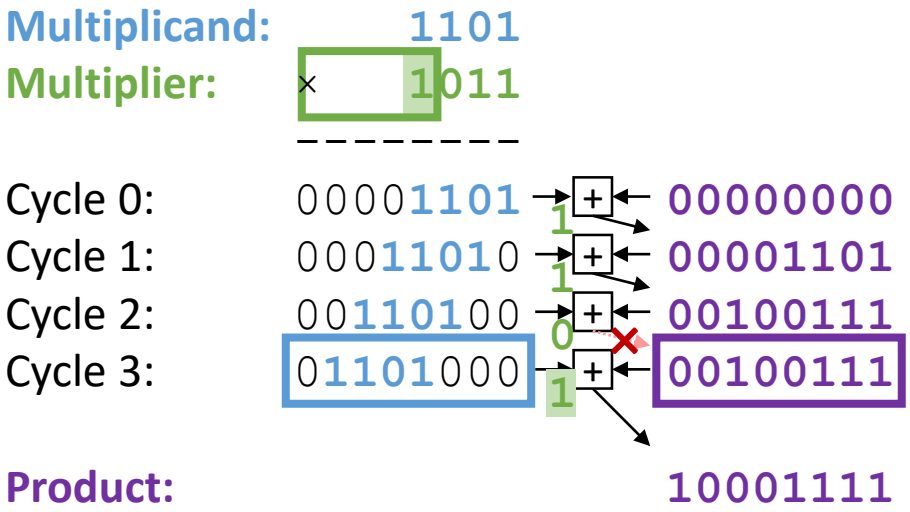


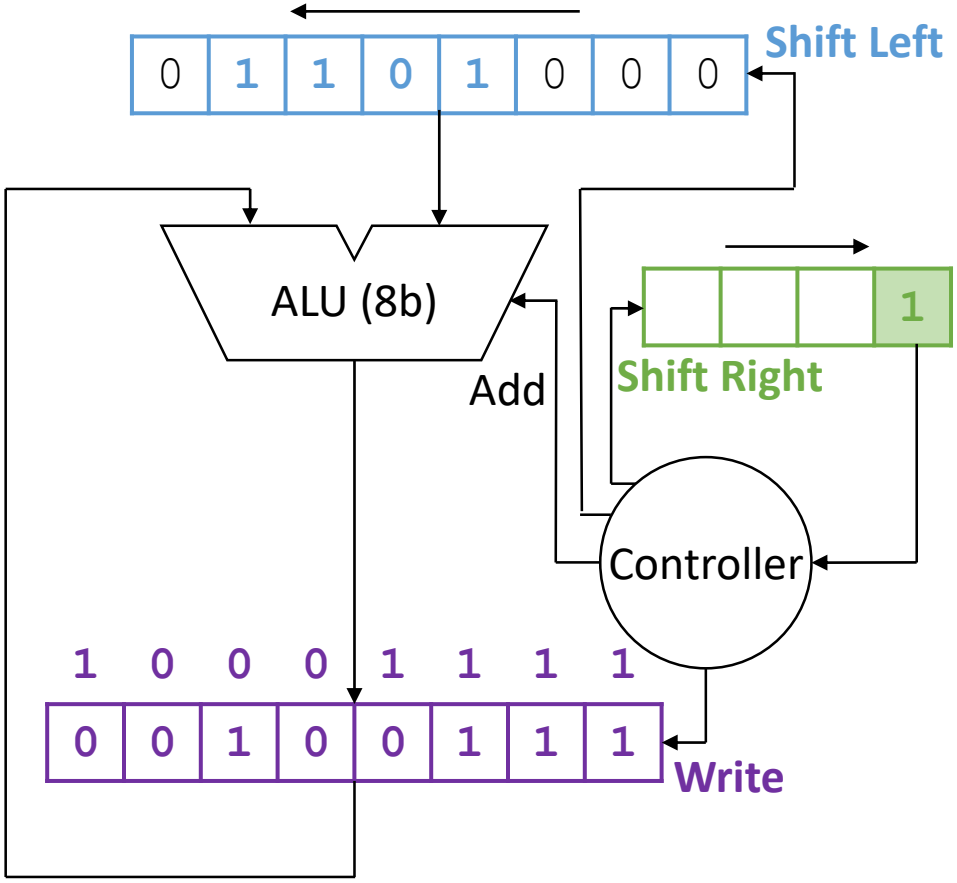


Overall approach:

- Shift **multiplicand** left on every cycle and add it to **product**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

Implementing Multiplication





Overall approach:

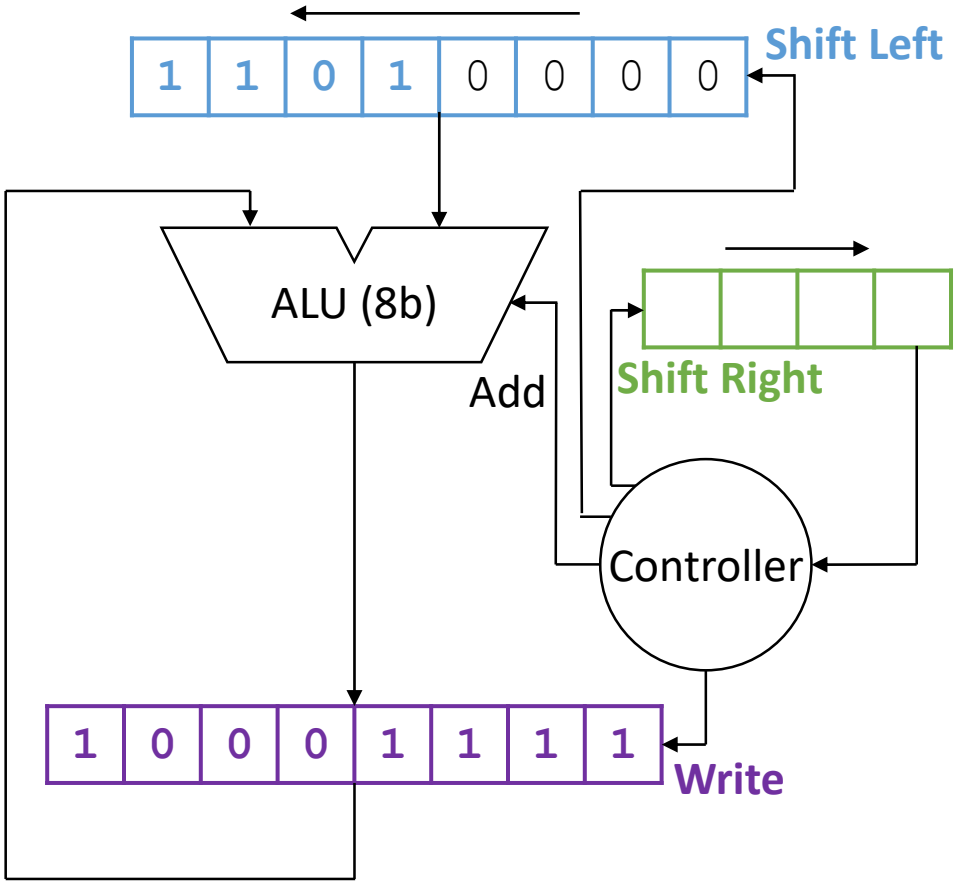
- Shift **multiplicand** left on every cycle and add it to **product**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

Implementing Multiplication

Multiplicand: 1101
Multiplier: × 1011

Cycle 0:	00001101	1	00000000
Cycle 1:	00011010	1	00001101
Cycle 2:	00110100	0	00100111
Cycle 3:	01101000	1	00100111

Product: 10001111



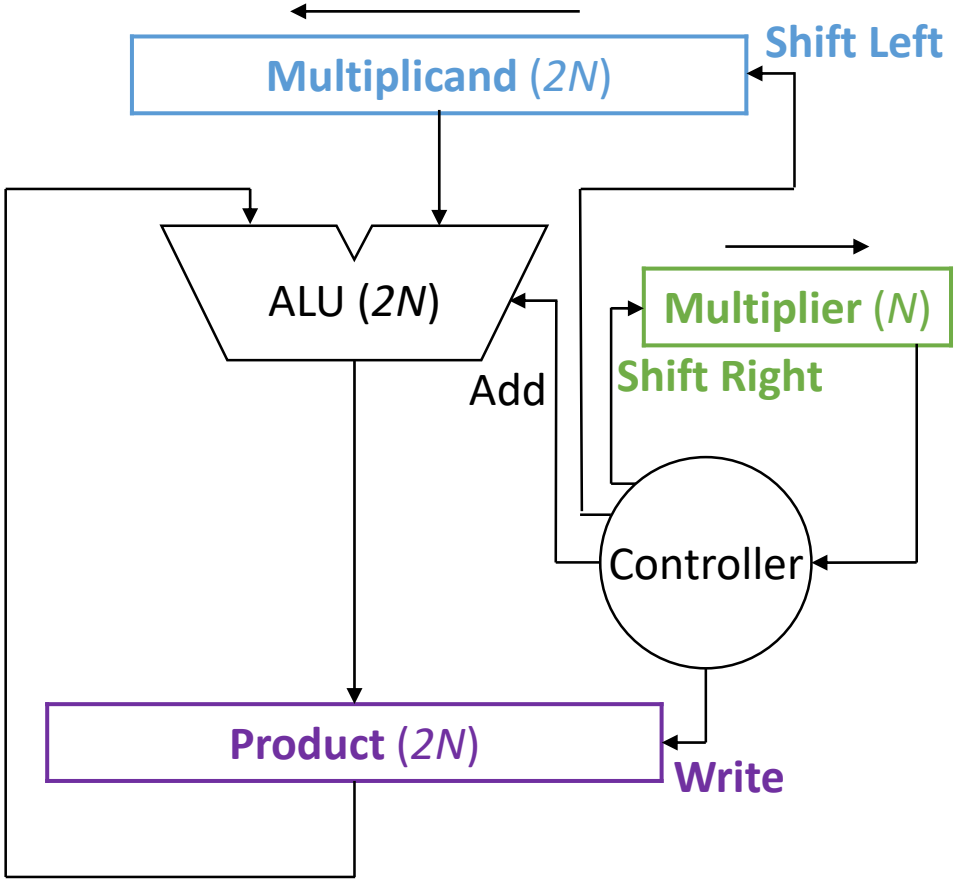
Overall approach:

- Shift **multiplicand** left on every cycle and add it to **product**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

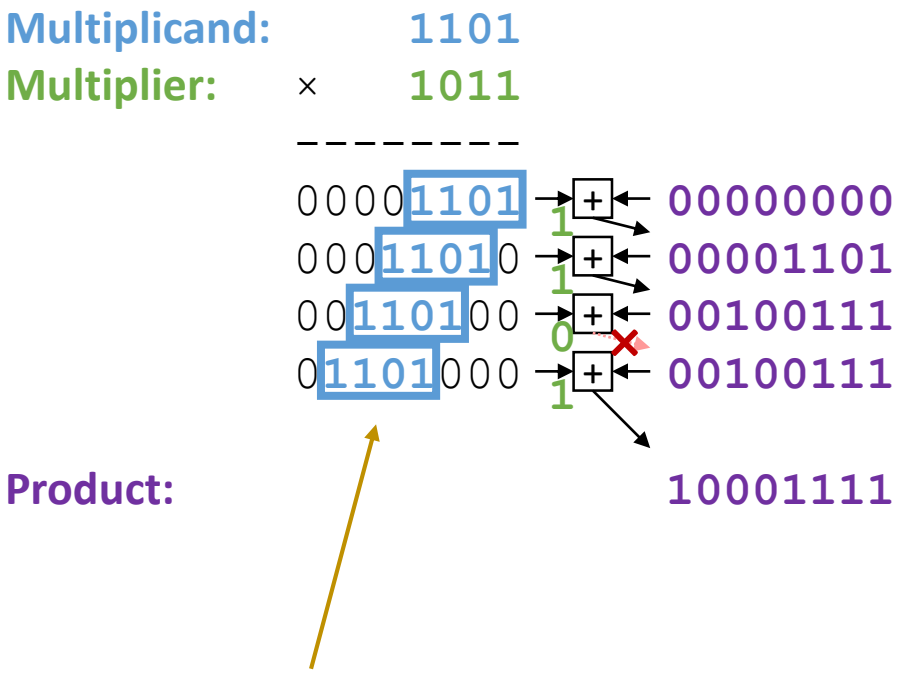
Cost of *N*-bit Multiplication

- Cost:
  - ALU: *2N*
  - Registers: *5N*
  - Time: *N*

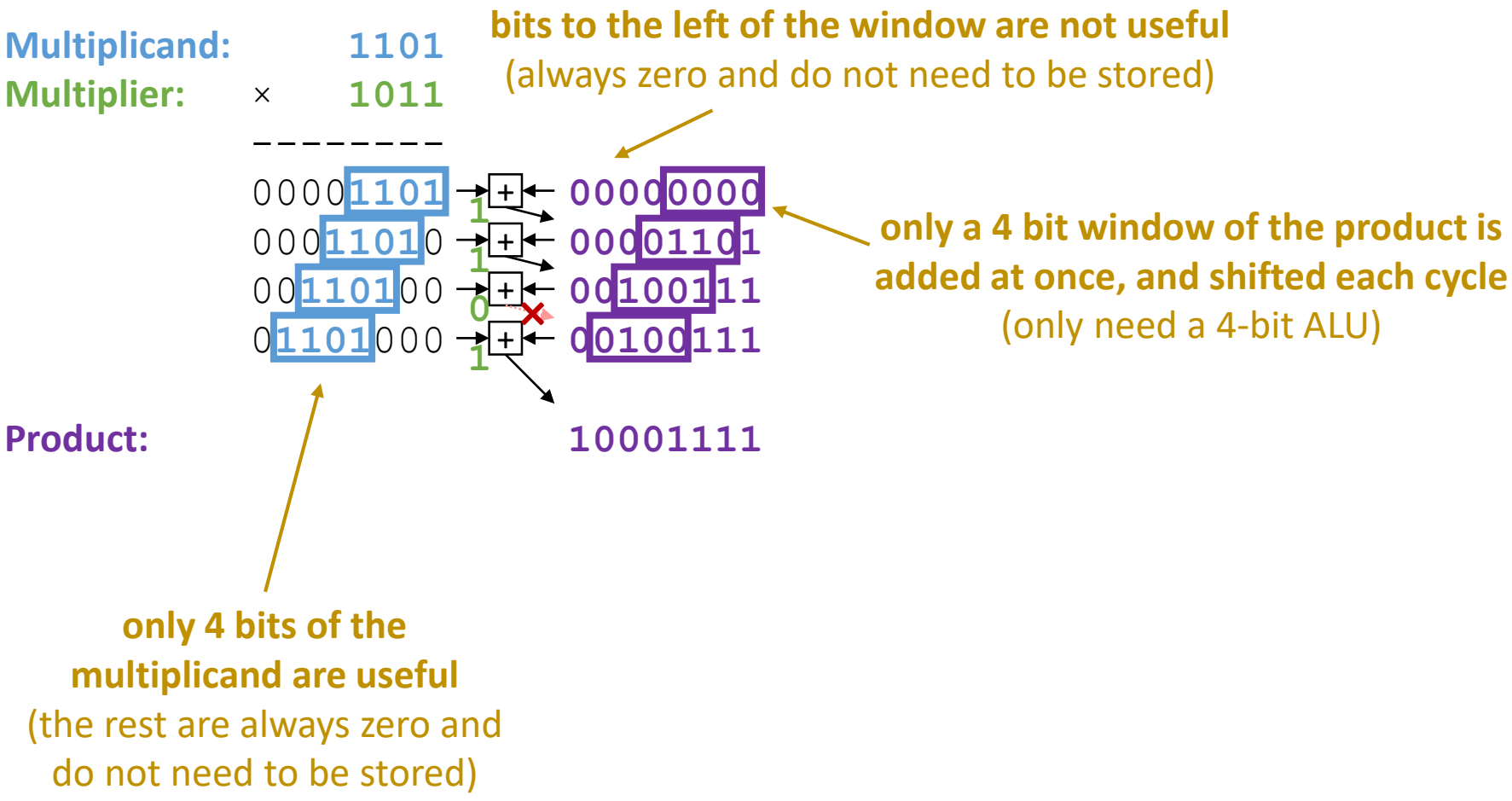
- Can we do better?
  - Use less hardware?
  - Finish faster?



Multiplication with Less Hardware

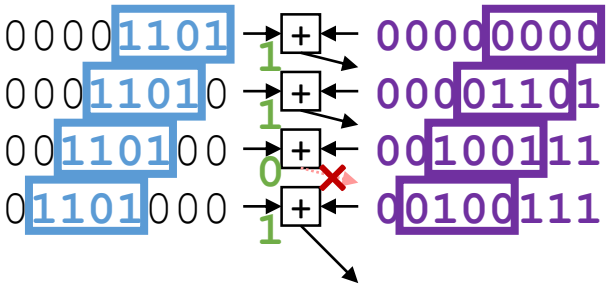


Multiplication with Less Hardware



Multiplication with Less Hardware

Multiplicand: 1101
Multiplier: × 1011



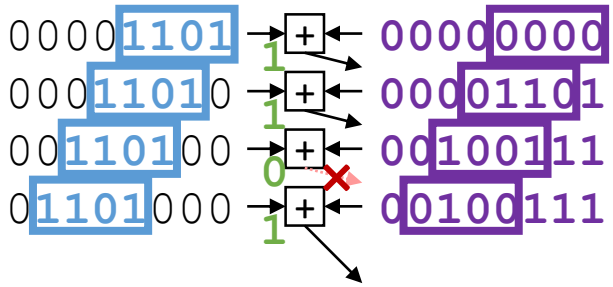
Product: 10001111

Overall approach:

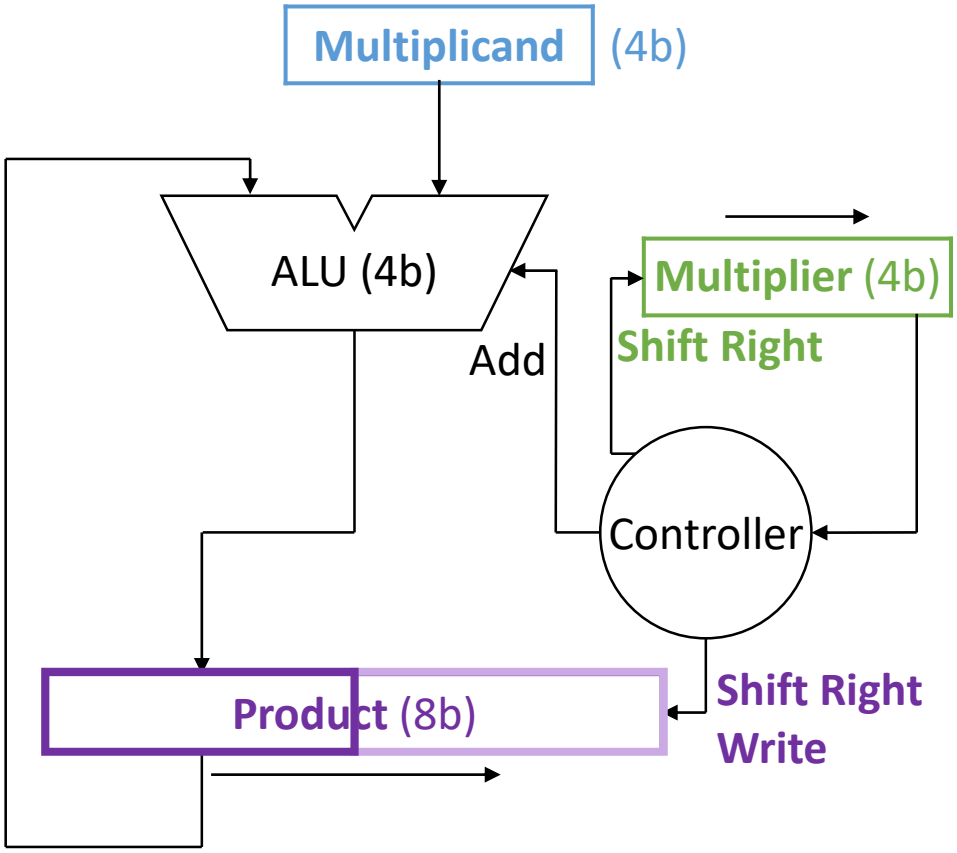
- Shift product window left (shift **product** right) on every cycle and add it to the stationary **multiplicand**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

Multiplication with Less Hardware

Multiplicand: 1101
Multiplier: × 1011



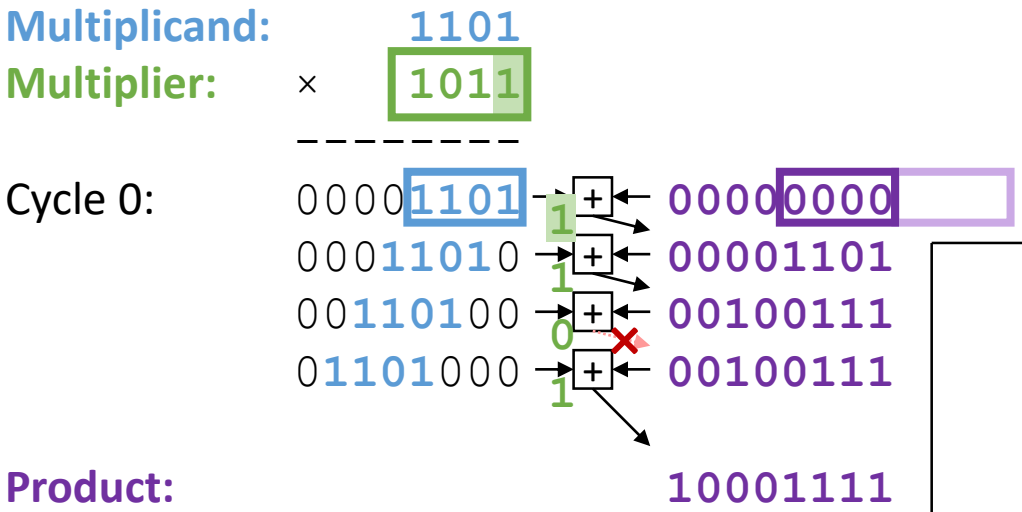
Product: 10001111

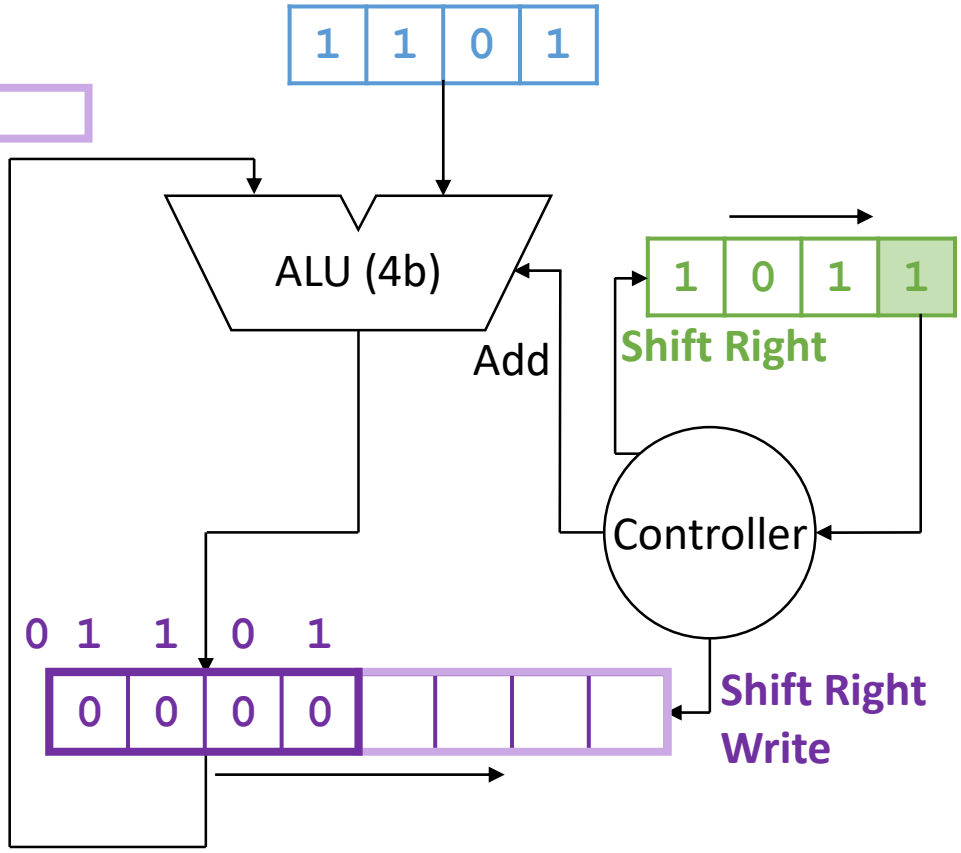


Overall approach:

- Shift product window left (shift **product** right) on every cycle and add it to the stationary **multiplicand**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

Multiplication with Less Hardware





Overall approach:

- Shift product window left (shift **product** right) on every cycle and add it to the stationary **multiplicand**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

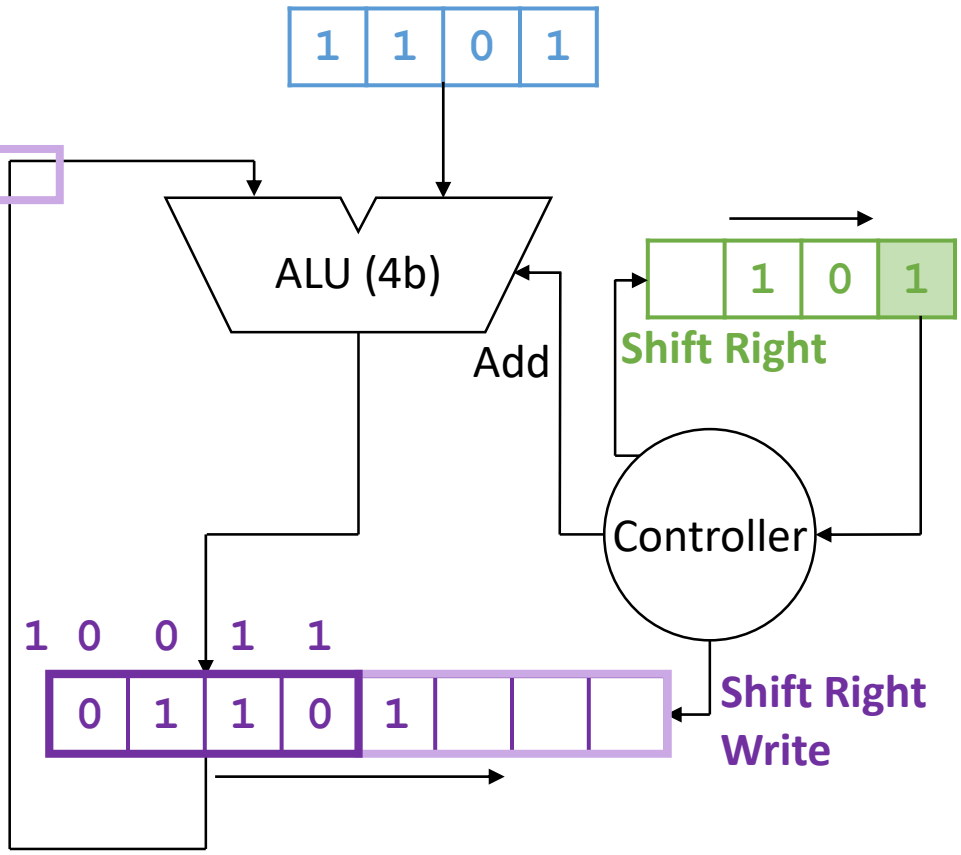
Multiplication with Less Hardware

Multiplicand: 1101
Multiplier: × 1011

Cycle 0:

00001101	1	00000000
000**1101**0	1	00001101
00**1101**00	0	00100111
0**1101**000	1	00100111

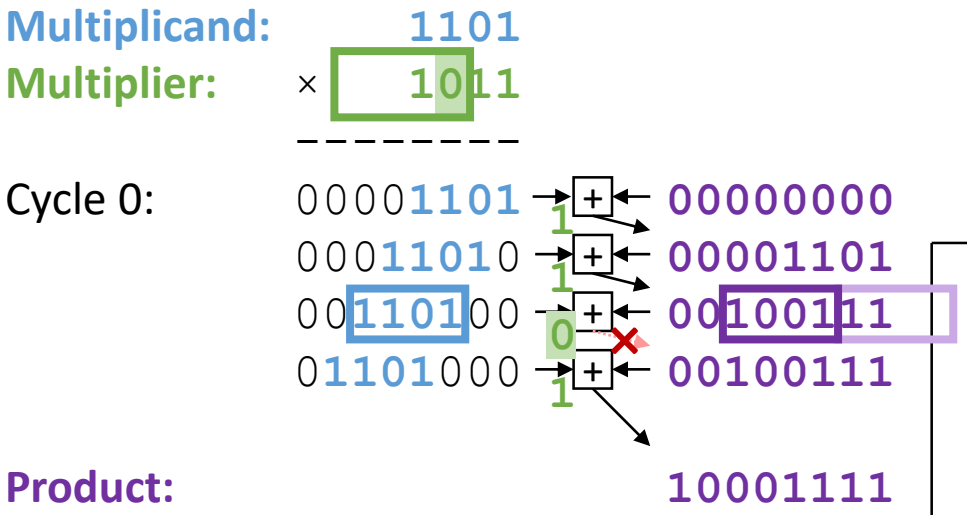
Product: 10001111

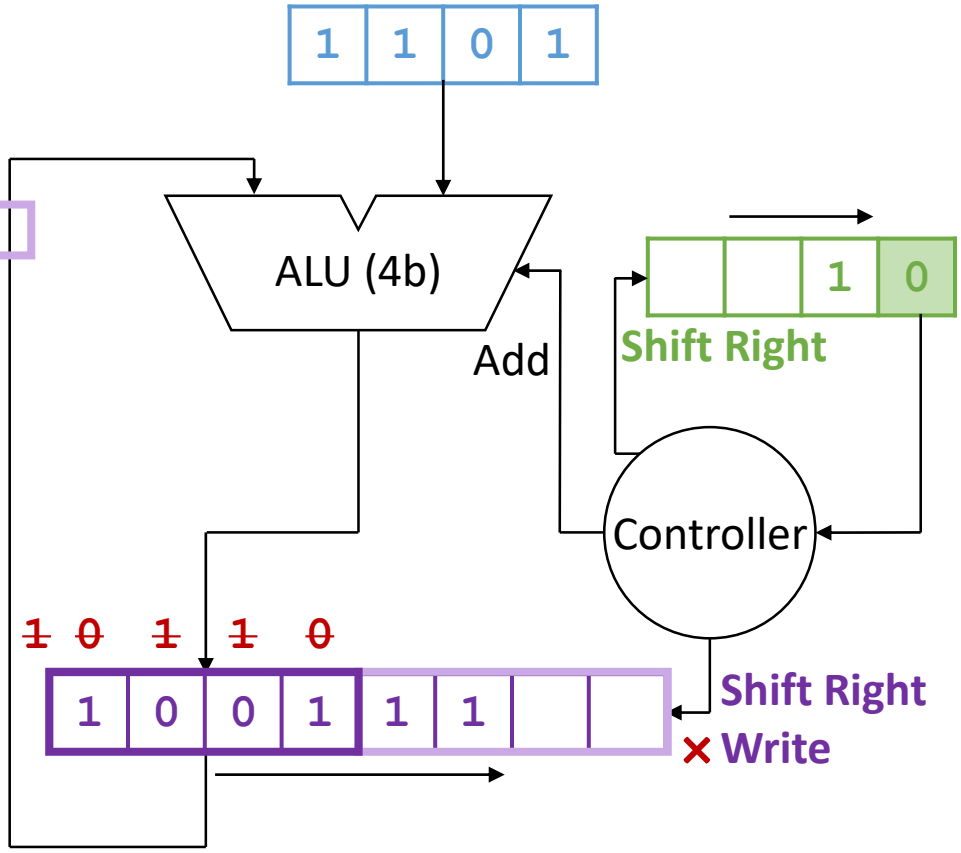


Overall approach:

- Shift product window left (shift **product** right) on every cycle and add it to the stationary **multiplicand**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

Multiplication with Less Hardware

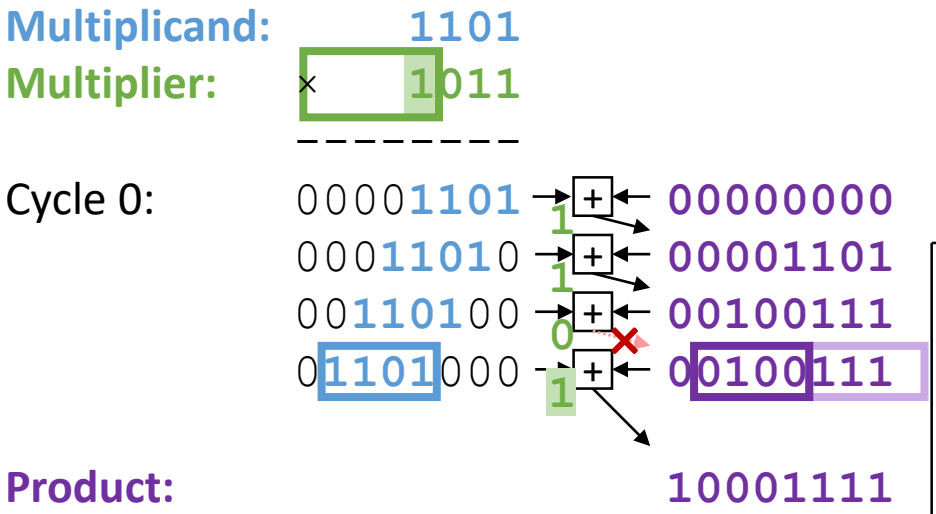


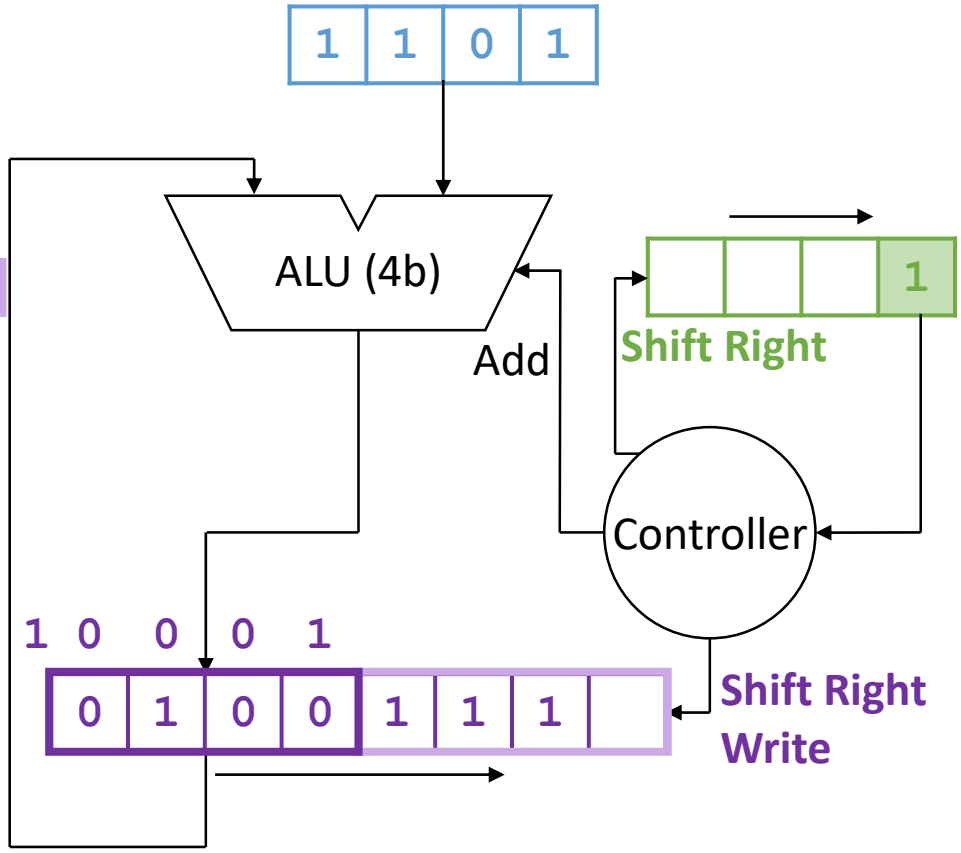


Overall approach:

- Shift product window left (shift **product** right) on every cycle and add it to the stationary **multiplicand**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

Multiplication with Less Hardware





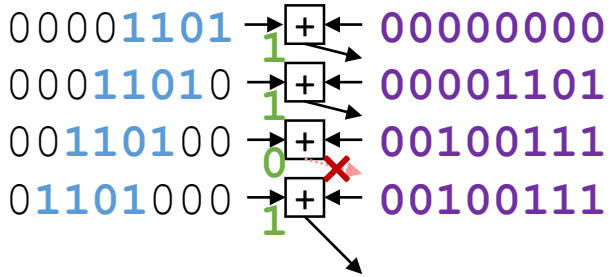
Overall approach:

- Shift product window left (shift **product** right) on every cycle and add it to the stationary **multiplicand**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

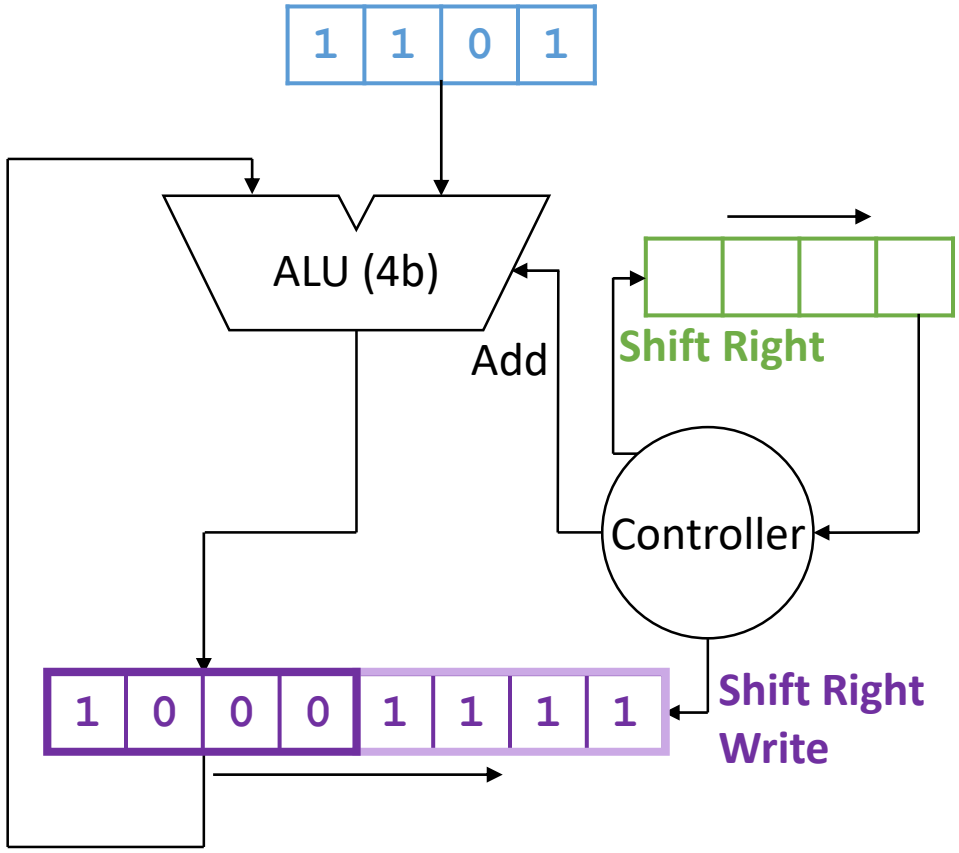
Multiplication with Less Hardware

Multiplicand: 1101
Multiplier: × 1011

Cycle 0:



Product: 10001111



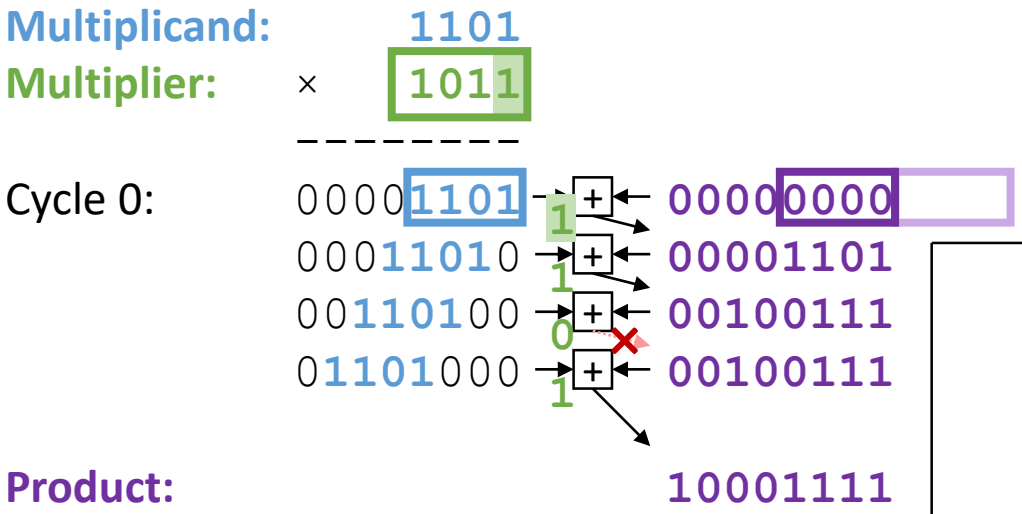
Overall approach:

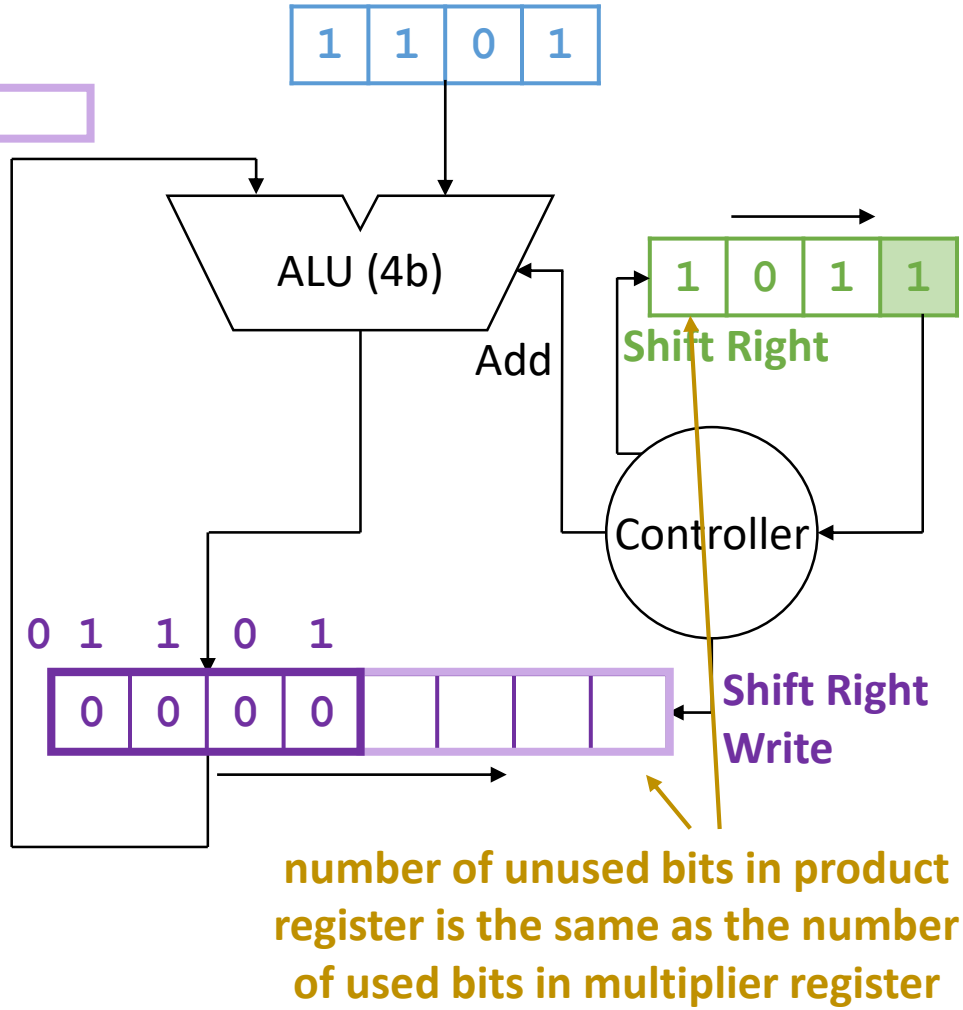
- Shift product window left (shift **product** right) on every cycle and add it to the stationary **multiplicand**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

Can we do even better?

Let’s replay and see…

Multiplication with Less Hardware

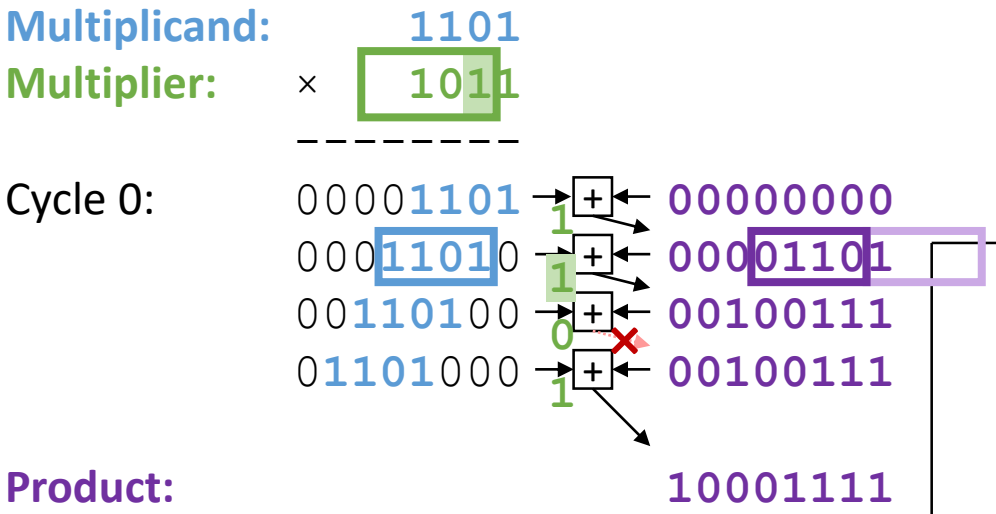


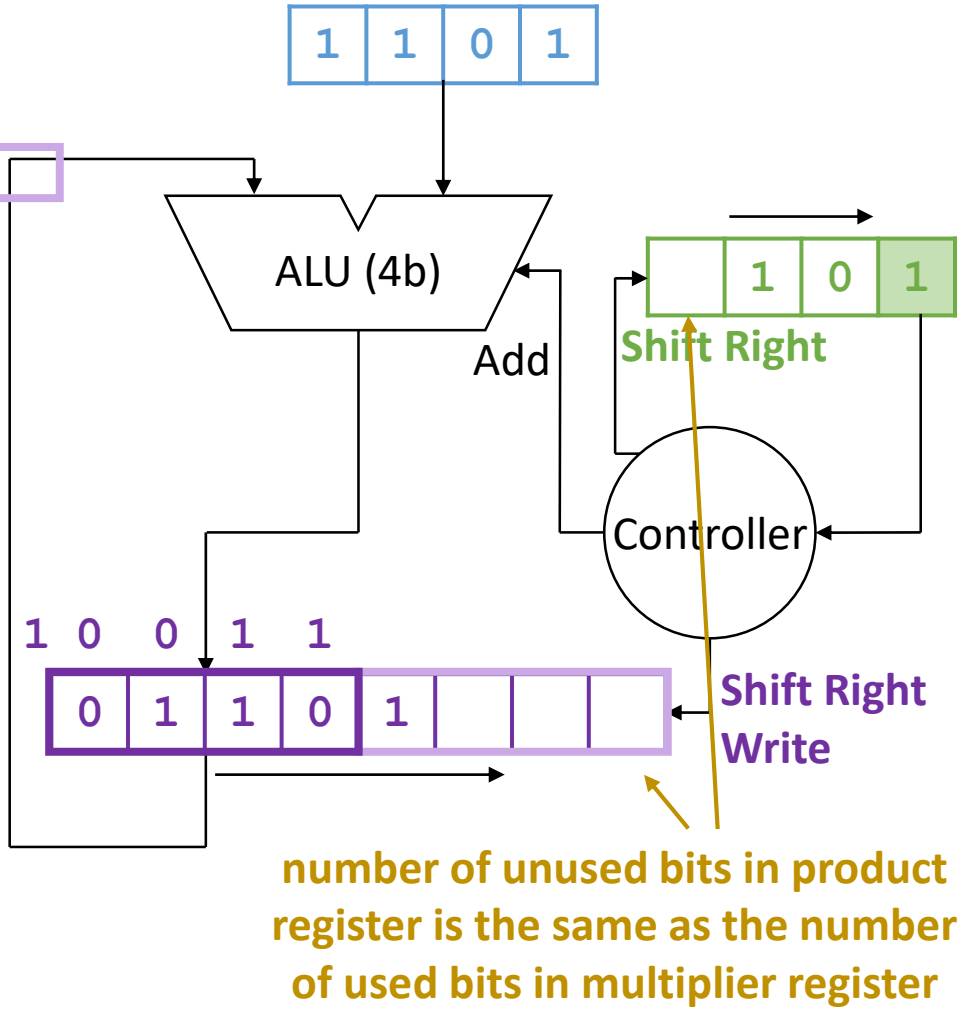


Overall approach:

- Shift product window left (shift **product** right) on every cycle and add it to the stationary **multiplicand**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

Multiplication with Less Hardware

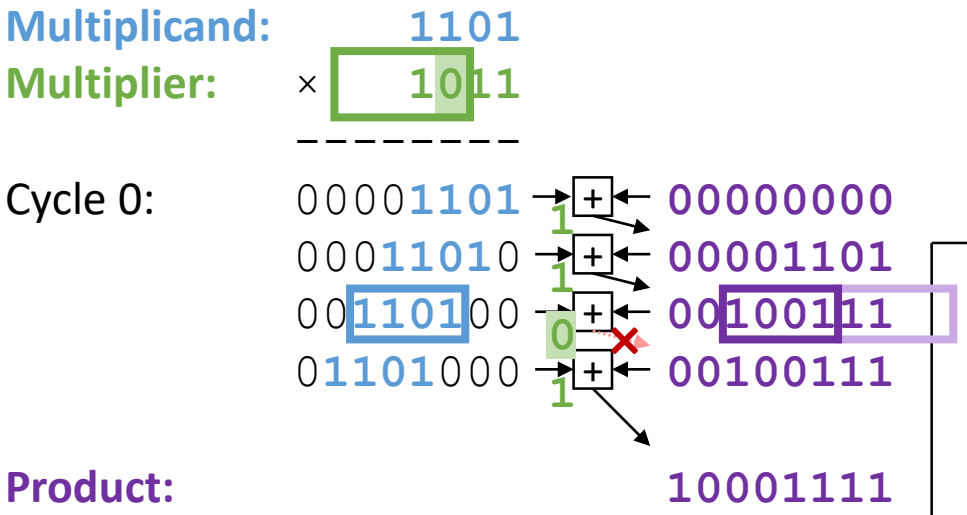


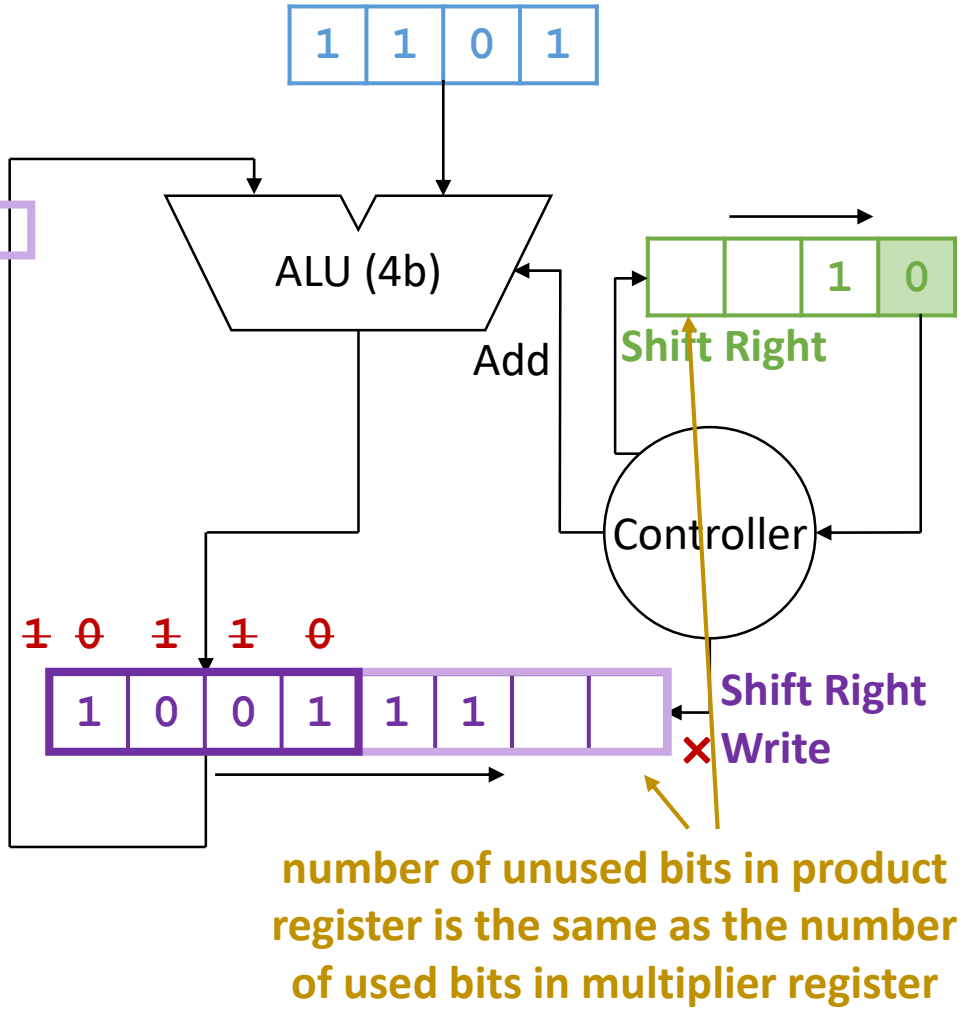


Overall approach:

- Shift product window left (shift **product** right) on every cycle and add it to the stationary **multiplicand**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

Multiplication with Less Hardware

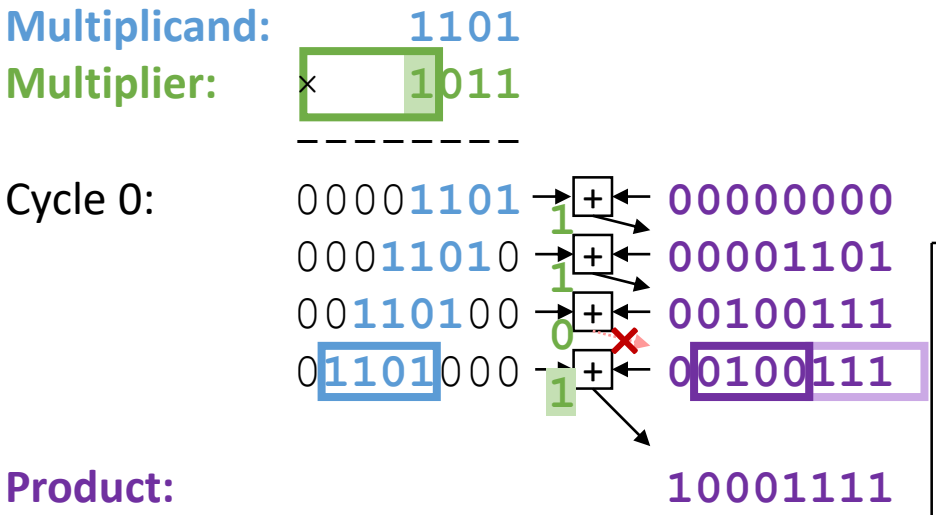


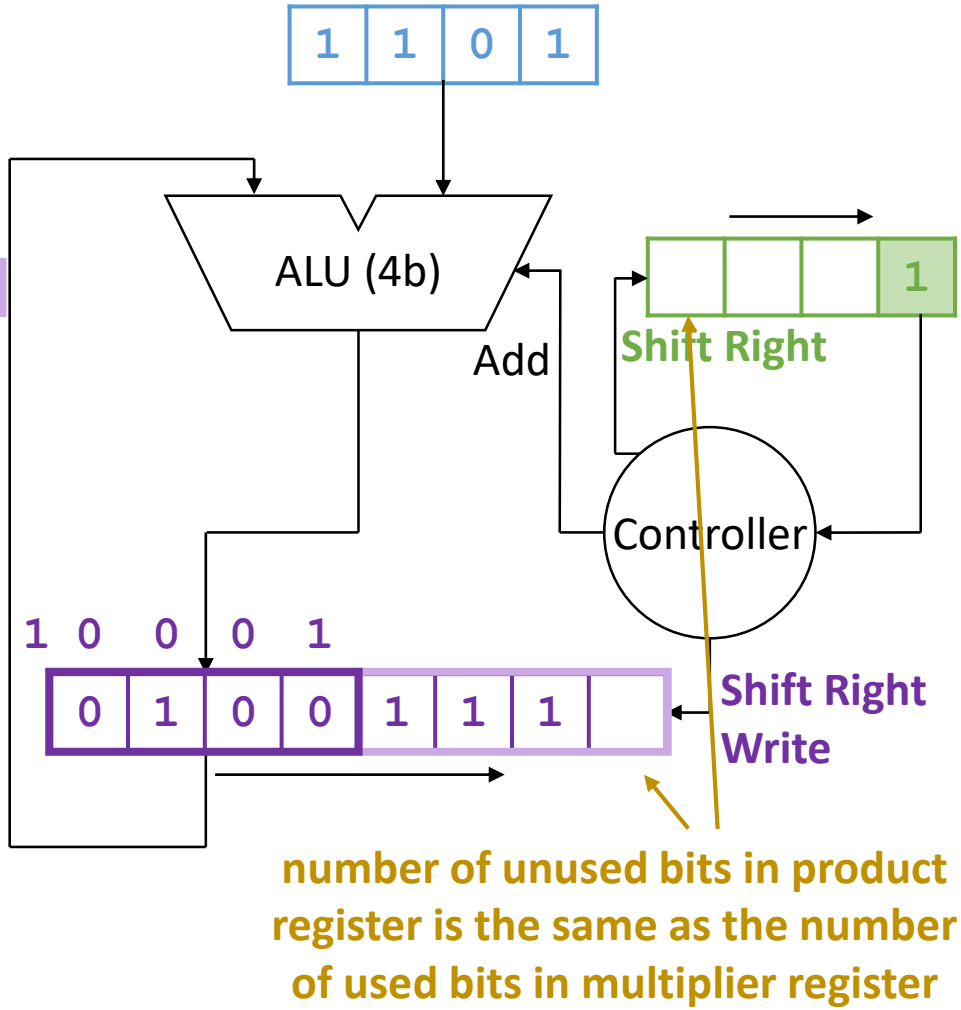


Overall approach:

- Shift product window left (shift product right) on every cycle and add it to the stationary multiplicand
- Use multiplier bit to choose whether or not to update the product with the sum

Multiplication with Less Hardware





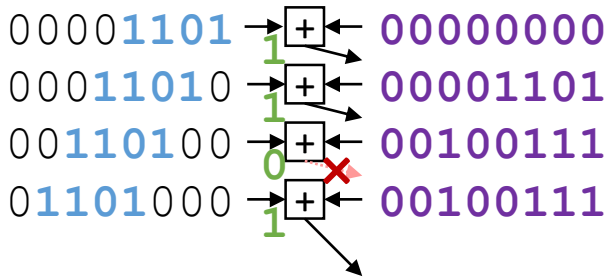
Overall approach:

- Shift product window left (shift **product** right) on every cycle and add it to the stationary **multiplicand**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

Multiplication with Less Hardware

Multiplicand: 1101
Multiplier: × 1011

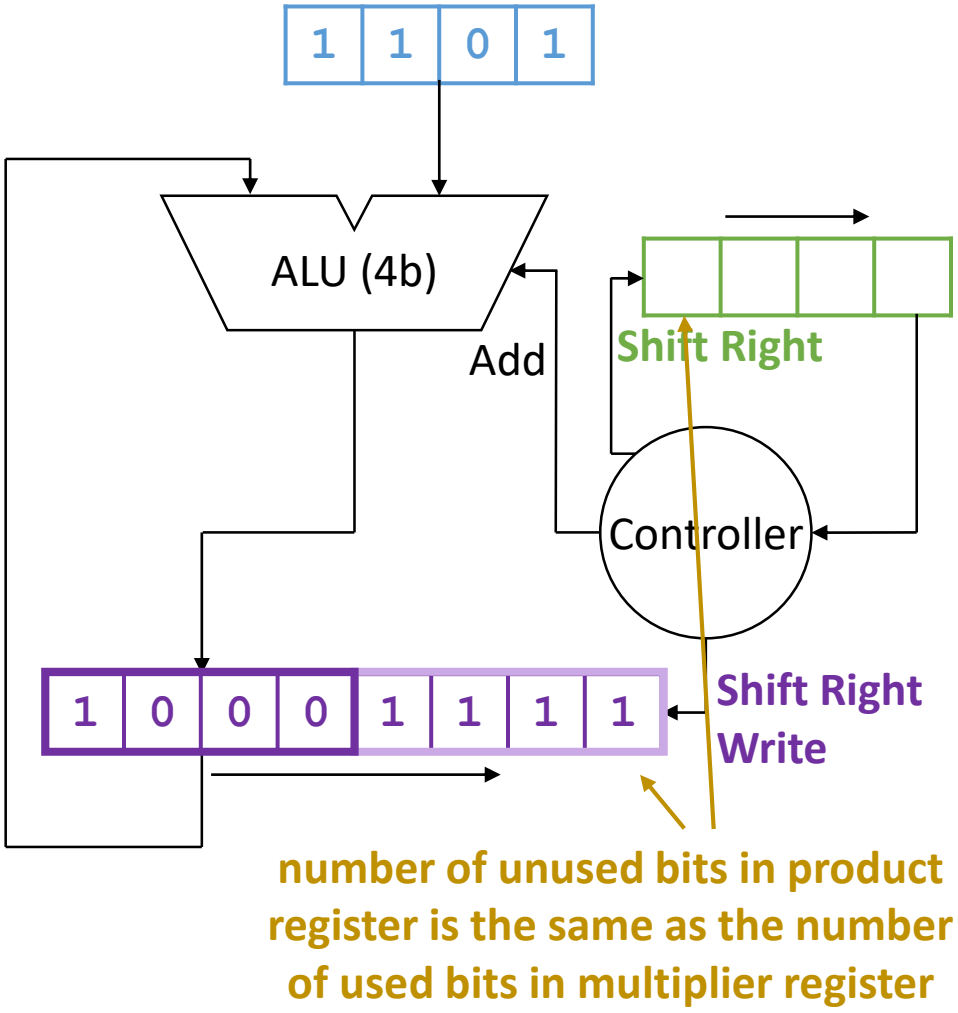
Cycle 0:



Product: 10001111

Overall approach:

- Shift product window left (shift **product** right) on every cycle and add it to the stationary **multiplicand**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum



Optimization: reuse the same register for multiplier and lower half of product

Let’s replay again and see…

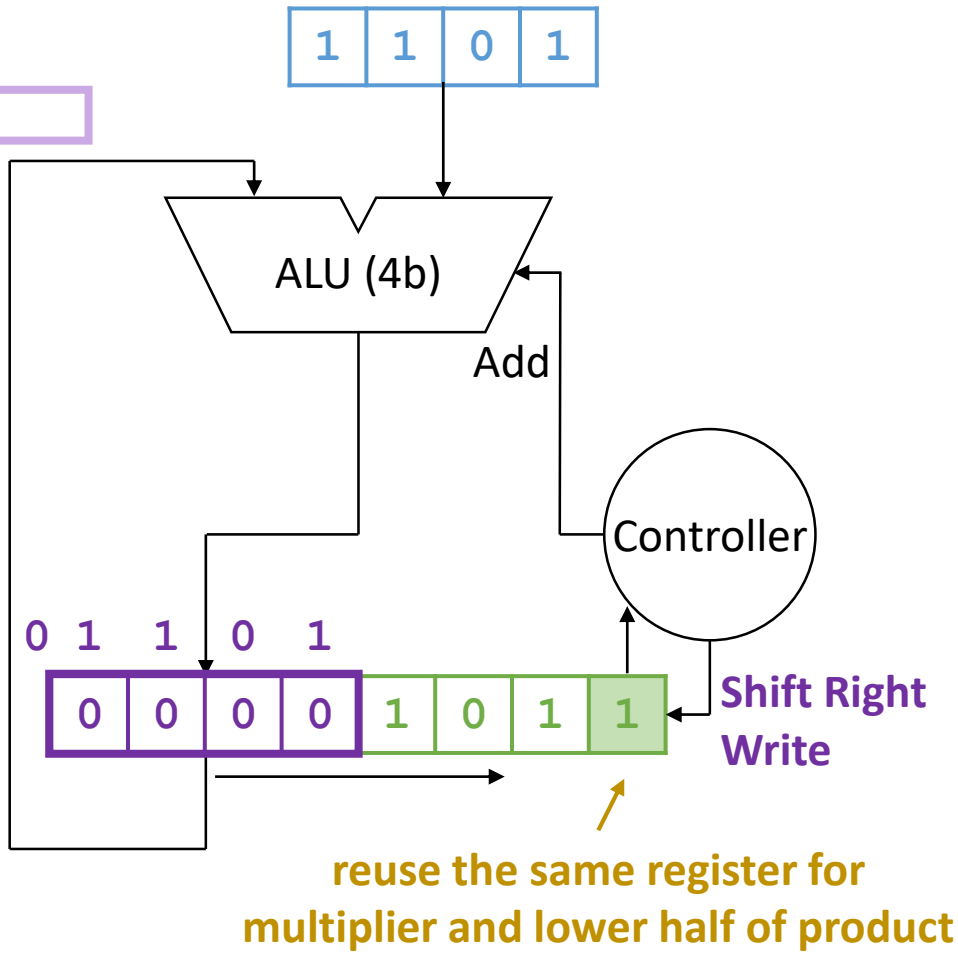
Multiplication with Less Hardware

Multiplicand: 1101
Multiplier: × 1011

Cycle 0:

0000**1101**	1	00000000
0001**1010**	1	00001101
0011**0100**	0	00100111
0110**1000**	1	00100111

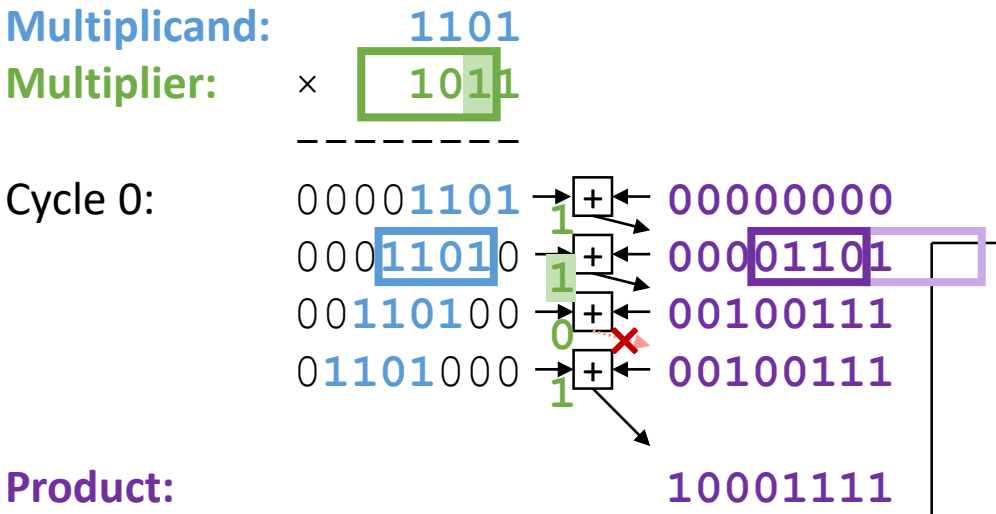
Product: 10001111

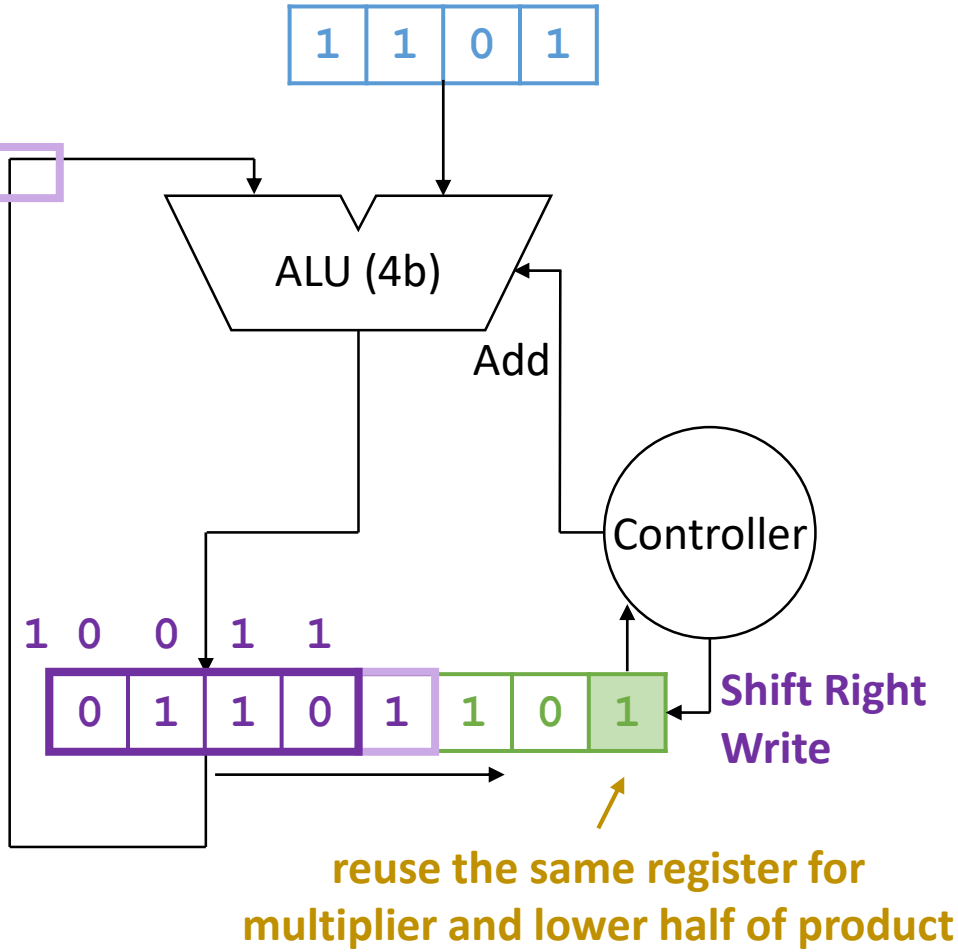


Overall approach:

- Shift product window left (shift **product** right) on every cycle and add it to the stationary **multiplicand**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

Multiplication with Less Hardware





Overall approach:

- Shift product window left (shift **product** right) on every cycle and add it to the stationary **multiplicand**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

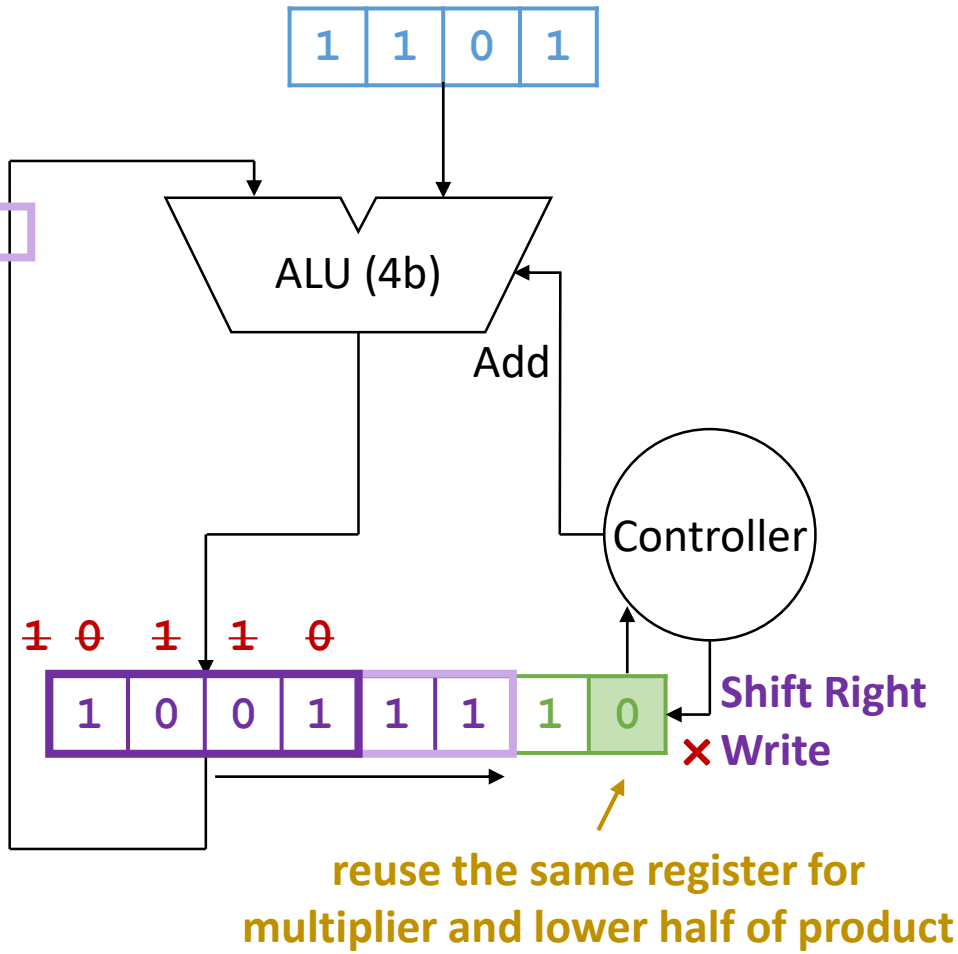
Multiplication with Less Hardware

Multiplicand: 1101
Multiplier: × 1011

Cycle 0:

```
00001101  +  00000000
00011010  +  00001101
00110100  +  00100111
01101000  +  00100111
```

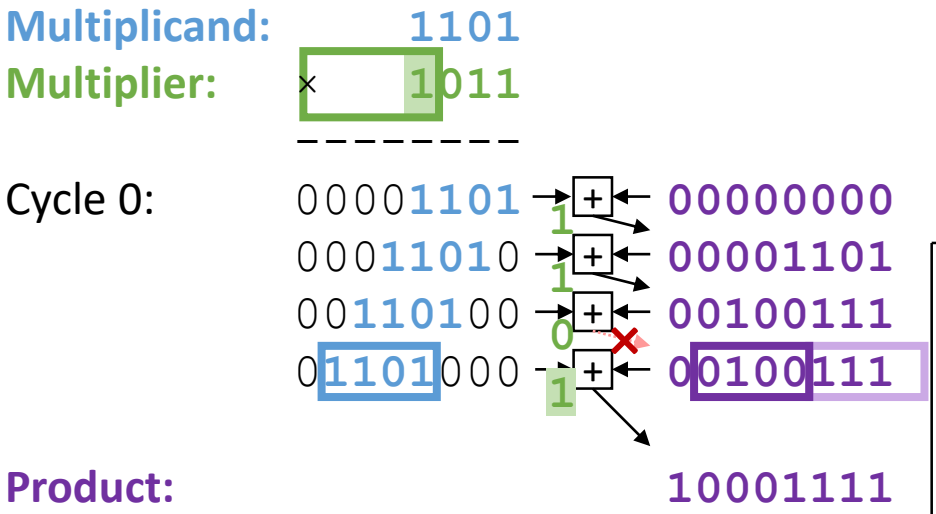
Product: 10001111

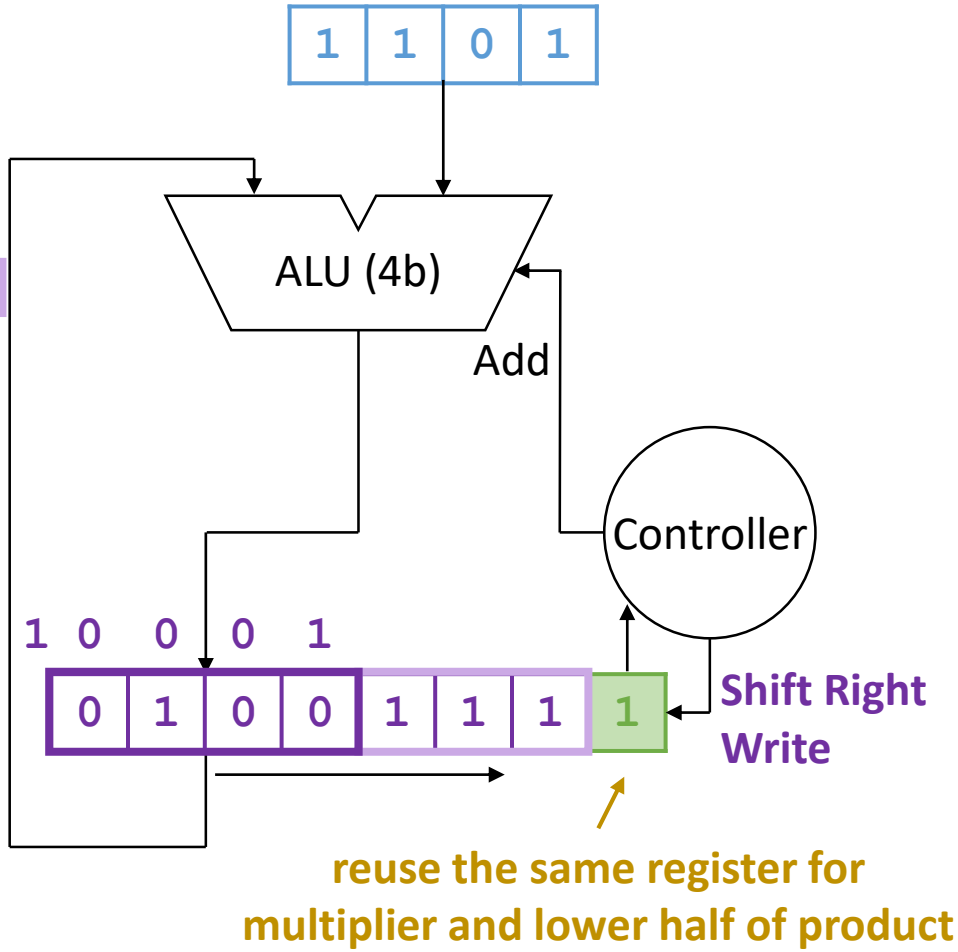


Overall approach:

- Shift product window left (shift **product** right) on every cycle and add it to the stationary **multiplicand**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

Multiplication with Less Hardware





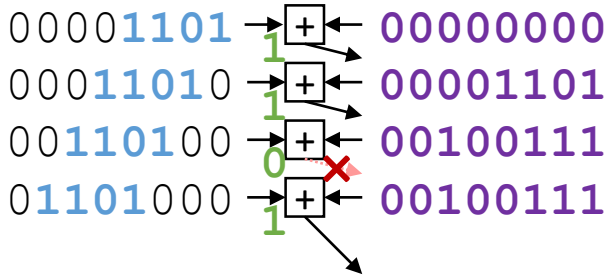
Overall approach:

- Shift product window left (shift **product** right) on every cycle and add it to the stationary **multiplicand**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum

Multiplication with Less Hardware

Multiplicand: 1101
Multiplier: × 1011

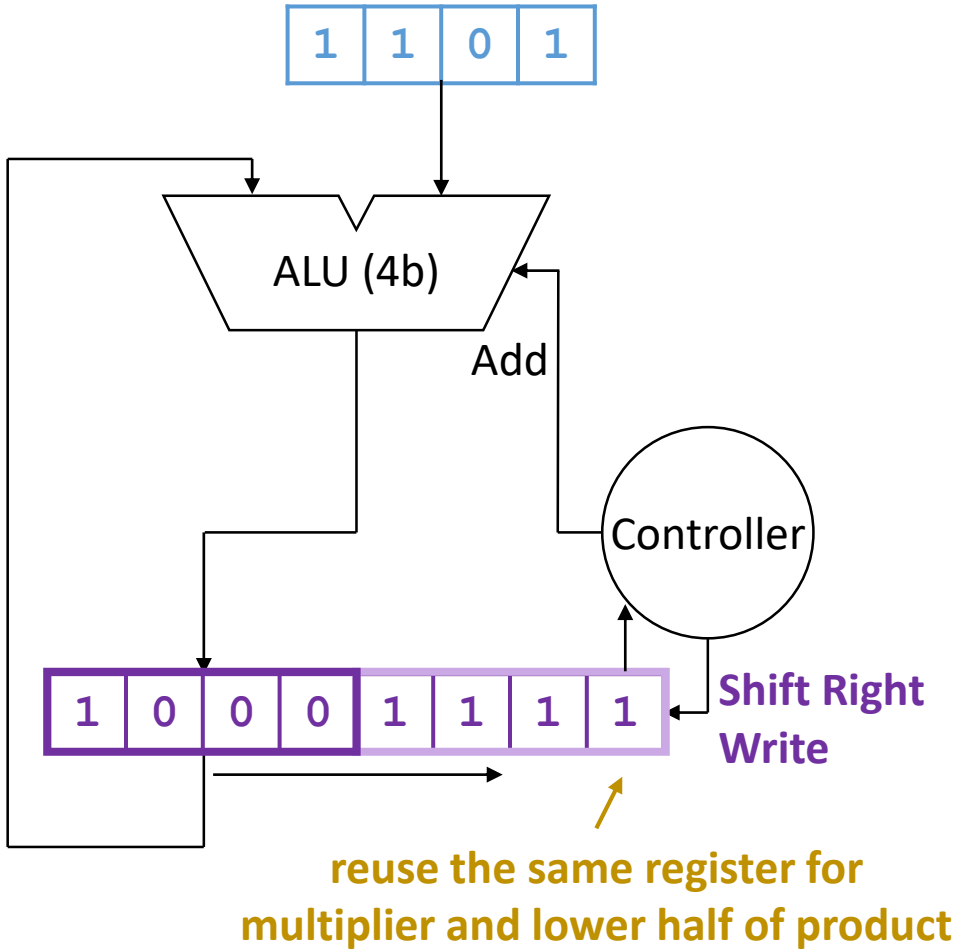
Cycle 0:



Product: 10001111

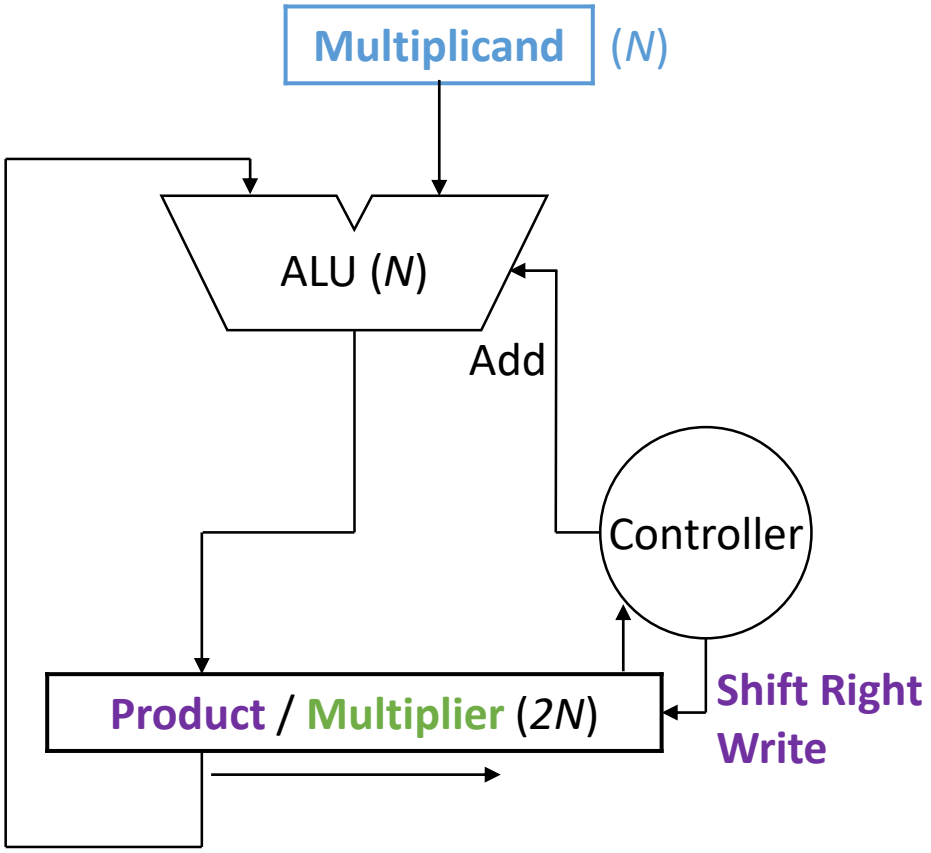
Overall approach:

- Shift product window left (shift **product** right) on every cycle and add it to the stationary **multiplicand**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum



Cost of *N*-bit Multiplication

- First design:
  - ALU: *2N*
  - Registers: *5N*
  - Time: *N*
- Second design:
  - ALU: *N*
  - Registers: *3N*
  - Time: *N*
- Can we do better?
  - Finish faster?



Multiplication Faster

Multiplicand: 1101
Multiplier: × 1011

```
--------
00001101
00011010
00000000
01101000
```

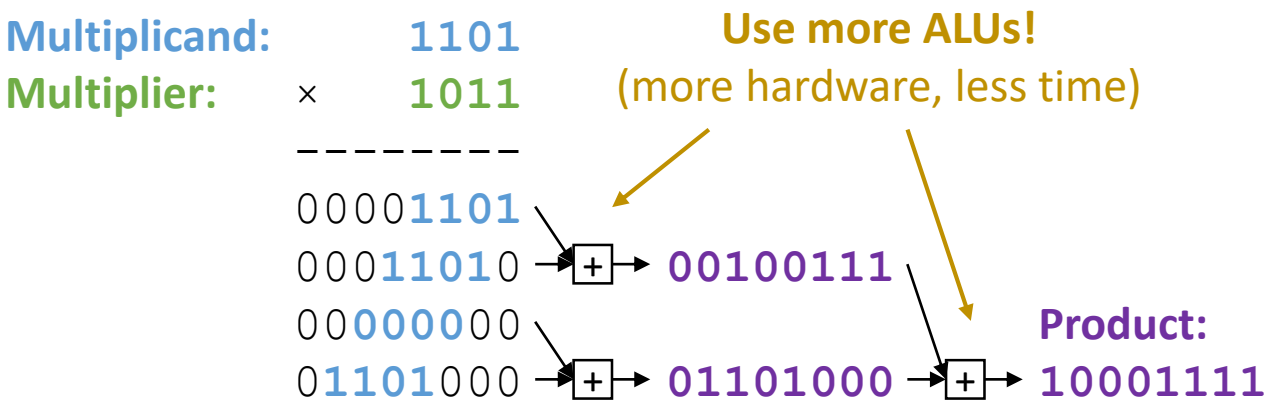
Multiplication Faster

Multiplicand: 1101

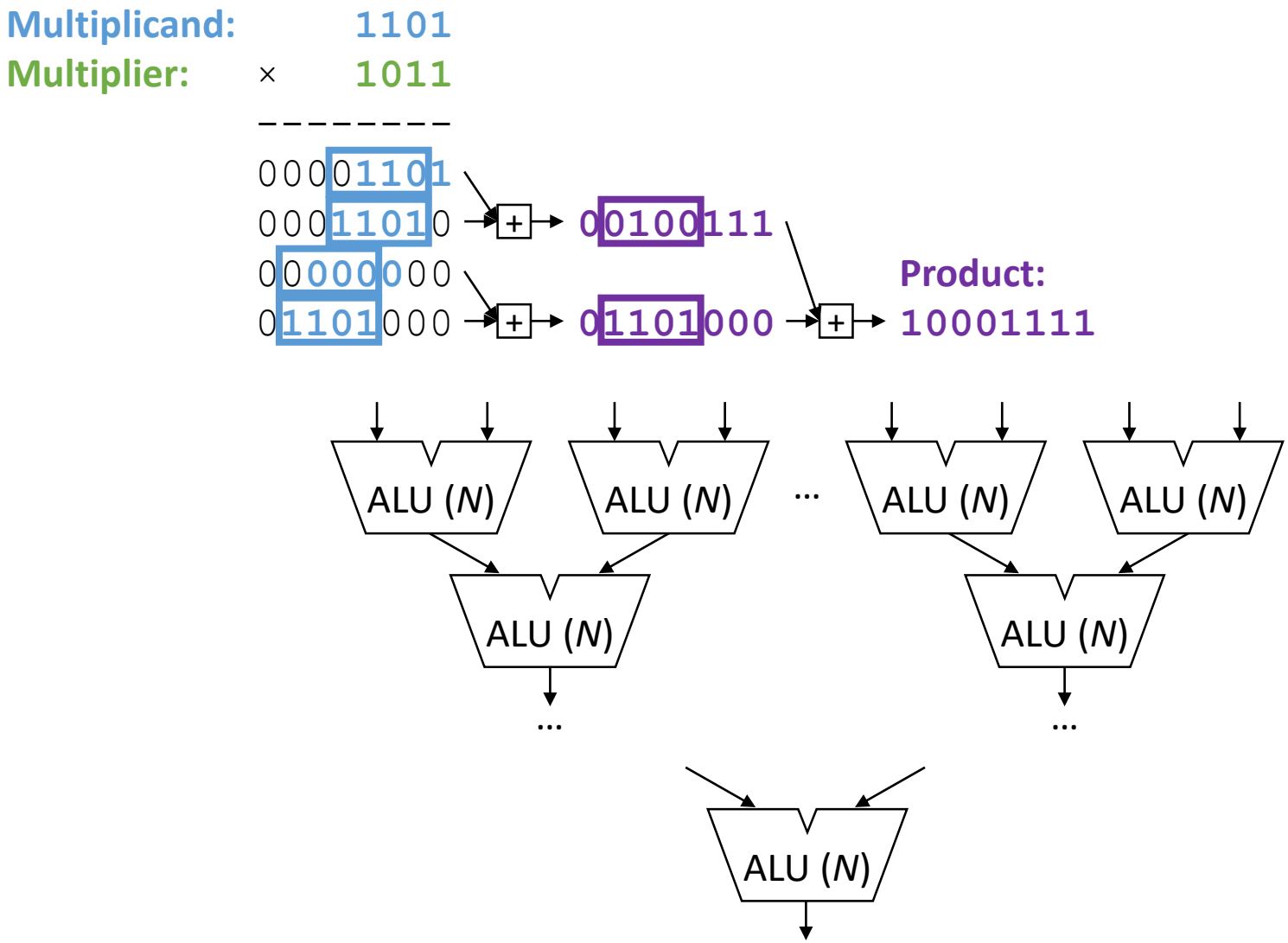
Multiplier: × 1011

```
--------
00001101
000110100  +  00100111
00000000
011010000  +  01101000
```

Multiplication Faster

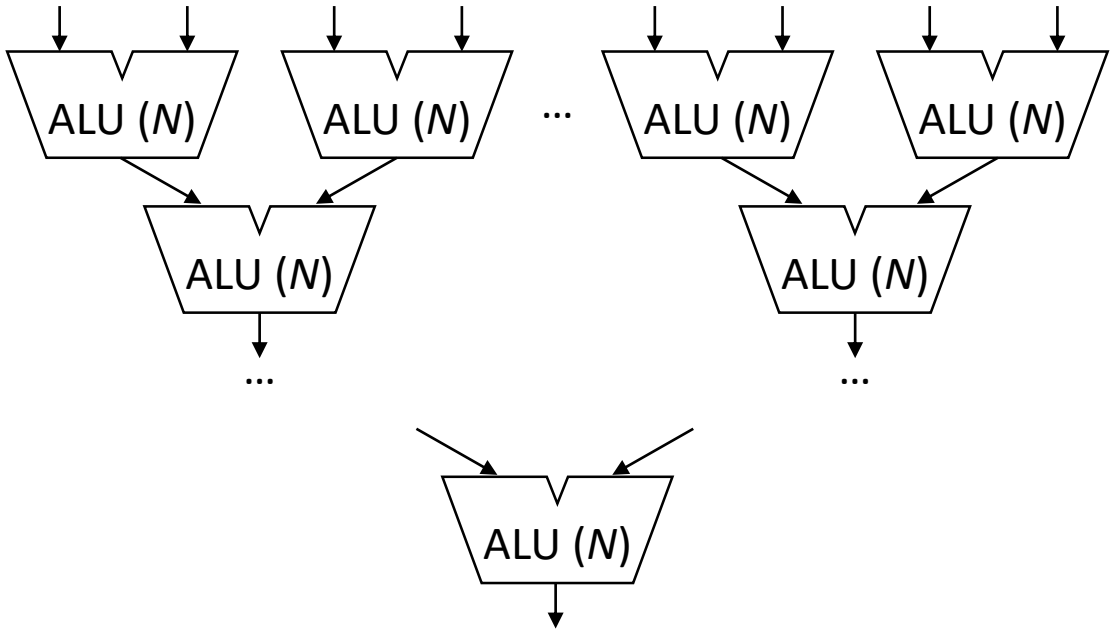


Multiplication Faster



Cost of *N*-bit Multiplication

- First design:
  - ALU: *2N*
  - Registers: *5N*
  - Time: *N*
- Second design:
  - ALU: *N*
  - Registers: *3N*
  - Time: *N*
- Third design:
  - ALU: $N \times (N\text{-}1)$
  - Time: $\log_2 N$



Textbook Sections

- Some of the content in these slides corresponds to:
  - Textbook:
    - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
  - Sections:
    - 3.3